

GUIDE DU PROJET 9

Filière cacao et café

Du producteur au port : entrepôt de données, ETL, Machine Learning, NLP et tableau de bord

Projets ETL et décisionnel · RNCP39765 · BC05 · FM01, FM02, FM05

Sommaire

Tout ce qu'il faut savoir pour réaliser le projet 9 de bout en bout.

01	Comprendre le projet	3
02	Exigences du cahier des charges	5
03	Architecture et outils	6
04	Les 15 sources de données	8
05	Modéliser l'entrepôt	12
06	Construire l'ETL	16
07	Incrémental et déclenchement automatique	19
08	Les 25 indicateurs	23
09	Machine Learning et NLP	25
10	Le tableau de bord en 5 pages	28
11	Plan de réalisation	34
12	Soutenance et validation finale	36

Le projet en un coup d'œil

15	7	1,7 M	25	3 + 2
sources de données	formats différents	lignes visées	indicateurs sur 5 pages	ML et NLP

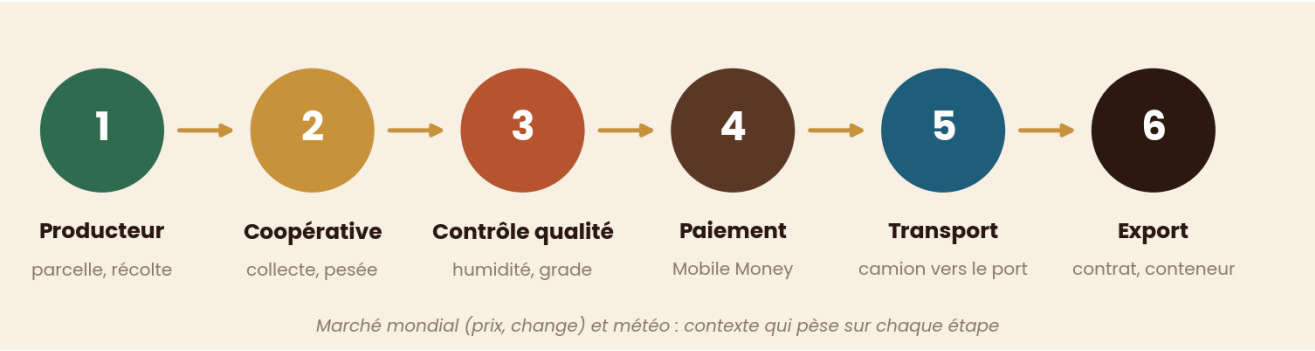
Comment utiliser ce guide

- **Chapitres 1 et 2** : comprendre ce qui est demandé et ce qui sera évalué.
- **Chapitres 3 à 7** : construire la partie technique, dans l'ordre (sources, entrepôt, ETL, incrémental, déclenchement).
- **Chapitres 8 à 10** : indicateurs, modèles d'IA et tableau de bord, qui donnent leur valeur aux données.
- **Chapitres 11 et 12** : organiser l'équipe, préparer la soutenance et vérifier la liste finale.
- Les extraits de code sont des **squelettes à adapter** à votre environnement : testez-les, ne les collez pas sans les lire.

01 Comprendre le projet

Une filière d'exportation, du producteur au port, vue à travers un entrepôt de données et un tableau de bord.

Le projet 9 vous place dans l'équipe data d'un exportateur de cacao et de café. La direction veut piloter la filière de bout en bout : combien est collecté, à quelle qualité, à quel prix, et avec quelle efficacité logistique jusqu'au port d'embarquement. Votre mission est de construire toute la chaîne décisionnelle : des sources hétérogènes, un processus ETL fiable, un entrepôt modélisé, des modèles de Machine Learning et de NLP, puis un tableau de bord de cinq pages.



La chaîne de valeur à piloter : six maillons, avec le marché mondial et la météo en arrière-plan.

Les questions que la direction se pose

Chaque page du tableau de bord répond à des questions précises. Gardez ce tableau sous les yeux : c'est lui qui justifie chaque table et chaque indicateur que vous allez construire.

Question de la direction	Page	Indicateurs et IA
Les quantités collectées progressent-elles, où, et avec combien de producteurs ?	2 · Production	6 à 10, prévision des tonnages (ML)
Quelles coopératives livrent des lots non conformes, et pour quelles raisons ?	3 · Qualité	11 à 15, prédiction de rejet (ML), types de défauts (NLP)
Le producteur est-il payé à un prix cohérent avec le marché, et assez vite ?	4 · Prix et finances	16 à 19
Quelle marge reste-t-il si le prix mondial, le change ou le transport varient ?	4 · Prix et finances	20, prévision de prix et simulation de marge (ML)
Où perd-on du temps, de l'argent et de la marchandise entre le champ et le port ?	5 · Logistique et export	21 à 25
Les contrats d'exportation sont-ils tenables (volumes, dates, pénalités) ?	5 · Logistique et export	Extraction des clauses (NLP)
Où en est la filière en un coup d'œil ?	1 · Synthèse	1 à 5

Contexte proposé et hypothèses de travail

Choix de contexte à valider avec votre enseignant

Le cahier des charges ne fixe pas de pays. Nous recommandons le Cameroun, qui donne du réalisme à toutes vos données générées : cacao dans le Centre, le Sud, le Sud-Ouest et l'Est ; café dans l'Ouest, le Nord-Ouest et le Littoral ; ports de Douala et de Kribi ; paiements par MTN MoMo et Orange Money ; monnaie FCFA (XAF). Tout le guide reste valable pour un autre pays : seuls les noms de régions, de ports et d'opérateurs changent.

Hypothèse	Valeur retenue
Produits	CACAO et CAFE (arabica et robusta) : un seul entrepôt, un attribut produit dans chaque fait
Campagne	Du 1er octobre au 30 septembre (calendrier commercial du cacao et du café), portée par dim_date
Historique généré	4 campagnes complètes, de 2022/23 à 2025/26, pour que les modèles de prévision aient de la matière
Devise de reporting	FCFA ; prix de marché en USD converti. Le FCFA (XAF) est rattaché à l'euro à 655,957 XAF pour 1 EUR : seul l'euro-dollar fluctue vraiment
Unités	Poids en kg, tonnage en tonnes, prix mondial en USD par tonne puis FCFA par kg, conteneurs en EVP
Fuseau horaire	UTC dans l'entrepôt, affichage Africa/Douala dans le tableau de bord

Vocabulaire express

Terme	Sens dans ce projet
Coopérative	Structure qui collecte les produits des planteurs, pèse, paie et revend au négociant ou à l'exportateur
Lot	Ensemble de sacs d'une même origine et d'une même période, analysé comme un tout au contrôle qualité
Grade	Classe de qualité d'un lot (grade 1, grade 2, hors grade) fixée selon l'humidité et les défauts
Incoterm	Règle de partage des coûts et des risques de transport : FOB (départ bord navire), CIF (coût, assurance, fret inclus), CFR
EVP	Équivalent vingt pieds : unité de compte des conteneurs maritimes
Filigrane	Valeur (date ou identifiant) mémorisée après un chargement pour ne lire ensuite que le nouveau
SCD de type 2	Dimension qui conserve l'historique : une ligne par version d'un producteur ou d'une coopérative

02 Exigences du cahier des charges

Ce que le document impose à tous les projets, et comment le projet 9 y répond point par point.

Le tableau suivant traduit chaque exigence commune en une décision concrète pour le projet 9 et en une preuve que vous devrez montrer à la soutenance. Cochez-le au fil de l'avancement.

Exigence	Notre réponse pour le projet 9	Preuve à montrer
15 sources, 4 formats au minimum	15 sources, 7 formats : CSV, Excel, JSON/API, PostgreSQL, GeoJSON, logs, XML. Huit sources sont générées	Schéma des sources, extraction réussie de chacune
1 million de lignes au moins	Environ 1,7 million de lignes visées. Générateurs paramétrables par un facteur d'échelle	Comptage par table après chargement
Entrepôt modélisé par vos soins	Constellation de 9 faits et 12 dimensions, dim_date, SCD2 sur producteur et coopérative	Matrice, schéma, scripts DDL
ETL : staging, qualité, rejets, reprise, journal	Schéma stg, 10 règles de qualité, table etl.rejet, table etl.run_log, tâches rejouables	Tables remplies, incident rejoué en direct
Au moins 2 mécanismes de déclenchement	A : planification Airflow. B : dépôt d'un fichier. Option C : déclencheur de base	DAG visibles, déclenchement en direct
Chargement incrémental	Filigrane, hachage, journal de suppressions. Run 0 complet puis runs 1 à 4	Comptages attendus et run_log
25 indicateurs en 5 pages	5 indicateurs par page, définitions formelles, requêtes SQL	Chapitre 8, tableau de bord
3 ML et 2 NLP	Prévision des tonnages, rejet de lot, prix et marge, défauts, contrats	Métriques d'évaluation affichées
Données textuelles	Observations du contrôle qualité et clauses des contrats (réalistes, variées)	Échantillon de textes, résultats NLP
Outil de visualisation	Metabase (ou Power BI), 5 pages, filtres communs, navigation, date de dernier chargement	Tableau de bord en démonstration
Livrables	Dossier de conception, scripts versionnés, entrepôt chargé, tableau de bord, démo, soutenance	Dépôt Git propre, support de soutenance

L'ordre de travail imposé par la logique du projet

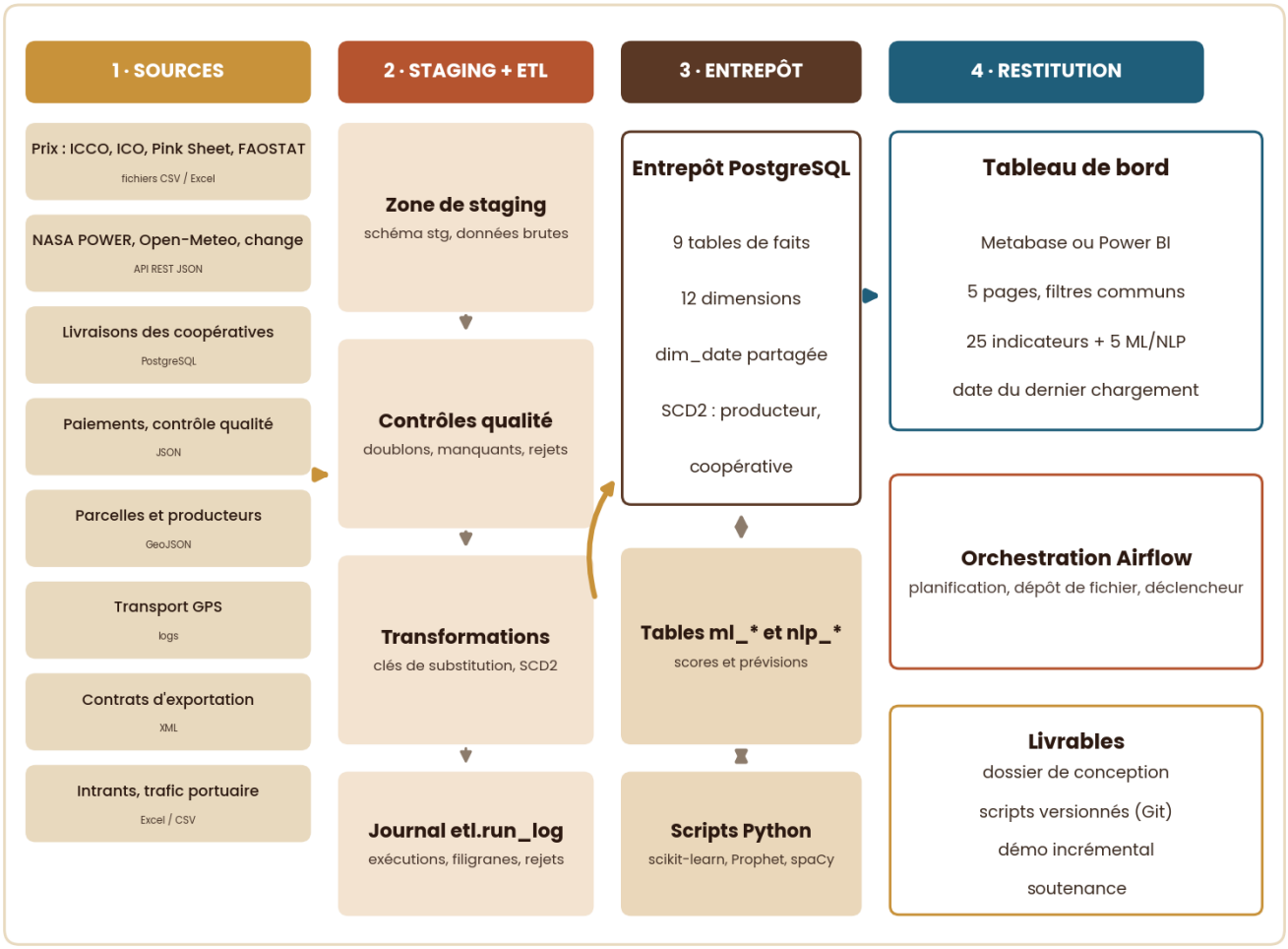
Le cahier des charges demande que l'entrepôt soit modélisé à partir des indicateurs. L'ordre est donc : indicateurs (chapitre 8) puis processus et matrice (chapitre 5) puis modèle de l'entrepôt puis flux ETL (chapitre 6). Ne commencez pas par les tables : commencez par les questions et les 25 indicateurs.

Les trois niveaux de réussite

Niveau	Contenu	Ce qui compte
Socle (indispensable)	Les 15 sources lues, entrepôt chargé, 25 indicateurs affichés, 2 déclenchements, un incrémental démontré	Tout fonctionne de bout en bout, même simplement
Solide	Qualité et rejets journalisés, reprise sur incident, SCD2 testé, 5 fonctionnalités ML et NLP avec métriques	La fiabilité et la démonstration reproductible
Remarquable	Déclencheur de base, tests automatisés, dossier de conception complet, tableau de bord soigné et commenté	La rigueur et la clarté de la présentation

03 Architecture et outils

Une architecture simple à faire tourner sur un ordinateur portable, et assez complète pour couvrir toutes les exigences.



Vue d'ensemble : sources, staging et ETL, entrepôt, restitution, orchestration.

Cinq principes de conception

- **Deux bases PostgreSQL distinctes** : une base source qui simule le système des coopératives, une base d'entrepôt. Cela rend l'extraction réelle.
- **Un dossier de dépôt** (data/inbox) où arrivent les fichiers JSON, XML, CSV et Excel, avec les dossiers archive et quarantaine à côté.
- **Le staging vit dans l'entrepôt**, dans un schéma stg qui reçoit les données brutes avant tout contrôle.
- **Le ML tourne hors de l'outil de visualisation** : des scripts Python écrivent leurs résultats dans les schémas ml et nlp, exigence explicite du cahier des charges.
- **Un seul orchestrateur** déclenche et enchaîne toutes les étapes, et journalise chaque exécution.

Outils recommandés

Couche	Choix recommandé	Alternative autorisée	Pourquoi
Base de données	PostgreSQL 16	SQL Server	Gratuit, complet (SCD2, JSONB, déclencheurs, pg_notify)
ETL	Python (pandas, SQLAlchemy, psycopg) et SQL	Talend ou Pentaho	Code versionnable et testable, logique identique dans les trois cas
Orchestration	Apache Airflow	cron, Talend ou Pentaho Scheduler, SQL Server Agent	Planification, capteurs de fichiers, reprise, interface de suivi
Génération de données	Python : Faker, NumPy, pandas	Mockaroo	Volume paramétrable, graine fixe, corrélations réalistes
ML et NLP	scikit-learn, Prophet ou statsmodels, spaCy	Hugging Face (CamemBERT)	Standard, documenté, rapide à entraîner sur un portable
Visualisation	Metabase	Power BI, Superset, Tableau Public	Installation Docker en minutes, onglets et filtres connectés
Versionnement	Git (dépôt privé)	—	Livrable exigé : scripts et jobs versionnés

Les schémas de l'entrepôt

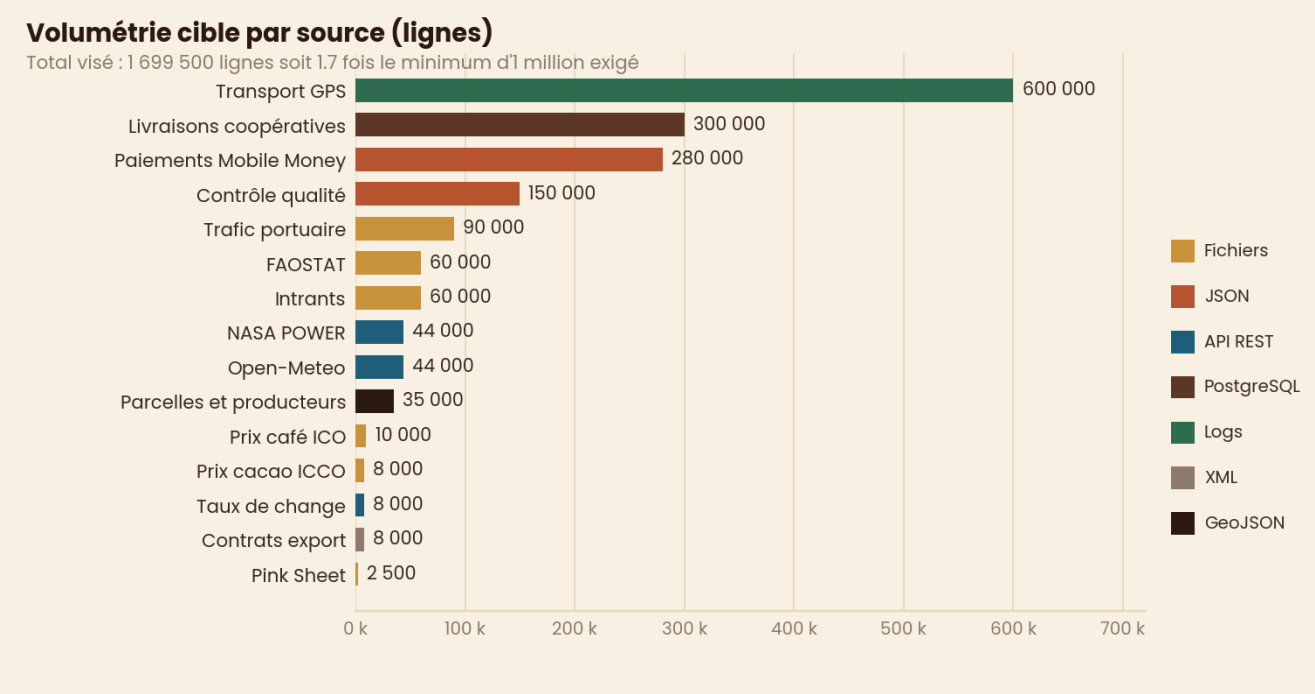
Schéma	Rôle	Contenu
`stg`	Zone de staging	Copies brutes des sources, une table par source, colonnes de traçabilité (run_id, charge_le)
`dwh`	Entrepôt	Dimensions et tables de faits : la seule source du tableau de bord pour les indicateurs
`etl`	Pilotage	run_log, watermark, rejet, fichier_traite : journalisation, filigranes, rejets
`ml`	Sorties du Machine Learning	Prévisions, scores, scénarios et métriques d'évaluation des modèles
`nlp`	Sorties du traitement du langage	Défauts classés, entités extraites des contrats, avec leur confiance

YAML · docker-compose.yml (squelette ; Airflow s'ajoute avec le fichier compose officiel du projet)

```yaml
services:
  pg_source:                    # simule la base des coopératives
    image: postgres:16
    environment: {POSTGRES_DB: coop_source, POSTGRES_USER: coop, POSTGRES_PASSWORD: changez_moi}
    ports: ["5433:5432"]
    volumes: ["./sql/source:/docker-entrypoint-initdb.d"]
  pg_dwh:                       # entrepôt : schémas stg, dwh, etl, ml, nlp
    image: postgres:16
    environment: {POSTGRES_DB: dwh, POSTGRES_USER: dwh, POSTGRES_PASSWORD: changez_moi}
    ports: ["5434:5432"]
    volumes: ["./sql/dwh:/docker-entrypoint-initdb.d", "dwh_data:/var/lib/postgresql/data"]
  metabase:
    image: metabase/metabase:latest
    ports: ["3000:3000"]
    depends_on: [pg_dwh]
volumes: {dwh_data: {}}
```

04 Les 15 sources de données

Sept sources publiques, huit sources générées par vous, au total environ 1,7 million de lignes.



Volumétrie cible : les logs GPS, les livraisons et les paiements portent l'essentiel du volume.

Les jeux publics et les API cités par le cahier des charges sont à vérifier avant utilisation : disponibilité, conditions d'accès et licences évoluent, et certaines API demandent une clé gratuite. Téléchargez une fois les fichiers, versionnez un échantillon, et gardez en cache les réponses d'API pour que votre démonstration ne dépende jamais d'internet.

Sources publiques (1 à 7)

N°	Source	Contenu et accès	Champs clés	Charge
1	**Prix du cacao (ICCO)**	Cotations quotidiennes du cacao (Londres et New York) publiées par l'Organisation internationale du cacao. Excel ou CSV à télécharger	date, marché, prix, devise	Dépôt de fichier, mise à jour mensuelle
2	**Prix du café (ICO)**	Prix indicatifs composites de l'Organisation internationale du café, par groupe (arabicas, robustas). CSV. Unité : cents US par livre	date, groupe, prix_cents_lb	Dépôt de fichier. Conversion : USD/t = cents/lb × 22,0462
3	**World Bank Pink Sheet**	Fichier Excel mensuel des prix des matières premières (cacao, café arabica, café robusta) de la Banque mondiale	mois, produit, prix_usd_kg (à pivoter en format long)	Dépôt de fichier, mensuel
4	**FAOSTAT cacao et café**	Production, superficie, rendement et commerce par pays et par année. CSV en téléchargement groupé	pays, produit, élément, année, valeur	Complet puis hachage de lignes, annuel
5	**NASA POWER**	API REST sans clé, point par point : température, précipitations, rayonnement journaliers. Point d'accès : power.larc.nasa.gov/api/temporal/daily/point (paramètres parameters, community=AG, latitude, longitude, start, end, format=JSON)	site, date, T2M, PRECTOTCORR	Fenêtre glissante de 7 jours, quotidien
6	**Open-Meteo**	API REST sans clé : historique (archive-api.open-meteo.com/v1/archive) et prévisions à quelques jours (api.open-meteo.com/v1/forecast)	site, date, température, pluie	Fenêtre glissante, quotidien
7	**Taux de change**	API REST JSON de taux de référence (ex. Frankfurter, taux de la BCE) pour EUR/USD et EUR/GBP, puis USD/XAF calculé via la parité fixe de 655,957 XAF pour 1 EUR	date, paire, taux	Dernier jour chargé + 1, quotidien

Même les sources publiques ont besoin d'un plan B

Prévoyez pour chaque API un fichier JSON de repli dans data/reference. Si l'appel échoue (réseau, quota), l'ETL lit le repli, écrit un avertissement dans run_log et continue : c'est exactement le « traitement d'erreur » que le jury attend.

Sources générées (8 à 15)

N°	Source	Format	Structure à générer	Volume
8	**Livraisons des coopératives**	PostgreSQL (schéma coop)	livraison(livraison_id, producteur_id, cooperative_id, parcelle_id, produit, date_livraison, poids_brut_kg, poids_net_kg, nb_sacs, updated_at) et coop.cdc_journal	300 000
9	**Paiements aux producteurs**	JSON, un fichier par jour	paiement_id, livraison_id, producteur_id, operateur (MTN_MOMO, ORANGE_MONEY, VIREMENT, ESPECES), montant_fcfa, frais_fcfa, statut, horodatage	280 000
10	**Contrôle qualité des lots**	JSON	lot_id, cooperative_id, date_reception, date_analyse, humidite_pct, nb_grains_comptes, grains_defectueux_pct, grade, decision, observation (texte libre)	150 000
11	**Parcelles et producteurs**	GeoJSON	Feature : géométrie (polygone ou point) et propriétés producteur_id, nom, sexe, annee_naissance, cooperative_id, culture, surface_ha, certification, region, maj_le. Plusieurs instantanés pour le SCD2	35 000
12	**Transport vers le port**	Logs texte	Un ping par ligne : horodatage, camion_id, lot_id, latitude, longitude, vitesse_kmh, evenement (DEPART, ARRET, ARRIVEE_PORT), poids_kg, notes d'incident	600 000
13	**Contrats d'exportation**	XML	contrat id : acheteur, incoterm, port, produit, volume, prix, devise, et un bloc clauses contenant du texte libre (dates, volumes, pénalités)	8 000 (clauses)
14	**Intrants distribués**	Excel, un classeur par mois	date, producteur_id, cooperative_id, type_intrant (engrais, fongicide, plants, insecticide), quantite, cout_fcfa	60 000
15	**Trafic portuaire**	CSV	escale_id, navire, port, arrivee_prevue, arrivee_reelle, chargement_prevu, chargement_reel, conteneurs_charges, attente_quai_h	90 000

Rendre les données générées crédibles

Un jury remarque tout de suite des données uniformes. Les règles suivantes donnent un jeu cohérent, exploitable par les modèles, et vérifiable.

- **Identifiants cohérents entre sources** : un livraison_id généré existe dans la base, dans le paiement et dans le lot. Générez d'abord les référentiels (producteurs, coopératives, parcelles), puis les événements.
- **Saisonnalité** : récolte principale d'octobre à décembre, récolte intermédiaire d'avril à juin, creux le reste de l'année. Les volumes suivent aussi la pluie des mois précédents.
- **Prix producteur lié au prix mondial** : en général entre 55 % et 75 % du prix mondial converti, avec du bruit, plus un écart par coopérative.
- **Qualité probabiliste** : la probabilité de rejet augmente avec l'humidité et le taux de défauts, sans jamais devenir une règle exacte (sinon le modèle de rejet est trivial).
- **Transport lié à la géographie** : la durée dépend de la distance coopérative vers port, la perte de poids de la durée et de la saison des pluies.
- **Graine aléatoire fixe** et paramètre --scale : même jeu à chaque exécution, volume réglable de 1 % à 100 %.

Anomalies à injecter volontairement

Un ETL qui n'a rien à nettoyer ne prouve rien. Injectez des défauts à des taux connus, de façon à pouvoir annoncer « 2 % de doublons détectés, 100 % rejetés » le jour de la soutenance.

Anomalie	Taux	Où	Règle qui doit la détecter
Doublons exacts	2 %	Paiements, livraisons	R01 doublon sur clé métier
Valeurs manquantes	1 à 2 %	poids_net_kg, producteur_id, humidite_pct	R02 non-nullité
Valeurs aberrantes	0,5 %	humidité de 85 %, poids négatif	R03 domaines de valeurs

Anomalie	Taux	Où	Règle qui doit la détecter
Dates en formats mixtes	3 %	JJ/MM/AAAA et ISO dans les JSON	R05 normalisation des dates
Clés étrangères orphelines	0,3 %	Paiement vers une livraison inconnue	R04 intégrité référentielle
Pings GPS impossibles	0,5 %	Position hors du pays, vitesse de 400 km/h	R08 contrôle de plausibilité

Python · generate/livraisons.py (extrait, volume réglable par --scale)

```python
import argparse
import numpy as np
import pandas as pd

def generer_livraisons(n: int, producteurs: pd.DataFrame, seed: int = 42) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    jours = pd.date_range("2022-10-01", "2026-09-30", freq="D")
    # saisonnalité : forte d'octobre à décembre, moyenne d'avril à juin
    poids = np.where(jours.month.isin([10, 11, 12]), 3.0, np.where(jours.month.isin([4, 5, 6]), 1.5, 1.0))
    dates = rng.choice(jours, size=n, p=poids / poids.sum())
    prod = producteurs.sample(n, replace=True, random_state=seed).reset_index(drop=True)
    net = np.clip(rng.lognormal(mean=4.6, sigma=0.6, size=n), 15, 1500).round(1)
    return pd.DataFrame({
        "livraison_id": [f"LIV-{i:08d}" for i in range(n)],
        "producteur_id": prod["producteur_id"],
        "cooperative_id": prod["cooperative_id"],
        "parcelle_id": prod["parcelle_id"],
        "produit": rng.choice(["CACAO", "CAFE"], size=n, p=[0.65, 0.35]),
        "date_livraison": dates,
        "poids_net_kg": net,
        "poids_brut_kg": (net * rng.uniform(1.02, 1.08, n)).round(1),
        "updated_at": dates,
    })

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--scale", type=float, default=1.0)      # 0.01 pour tester, 1.0 pour la version complète
    args = ap.parse_args()
    # producteurs = pd.read_parquet("data/reference/producteurs.parquet")
    # df = generer_livraisons(int(300_000 * args.scale), producteurs)
    # df.to_sql("livraison", engine_source, schema="coop", if_exists="append", index=False)
```

05 Modéliser l'entrepôt

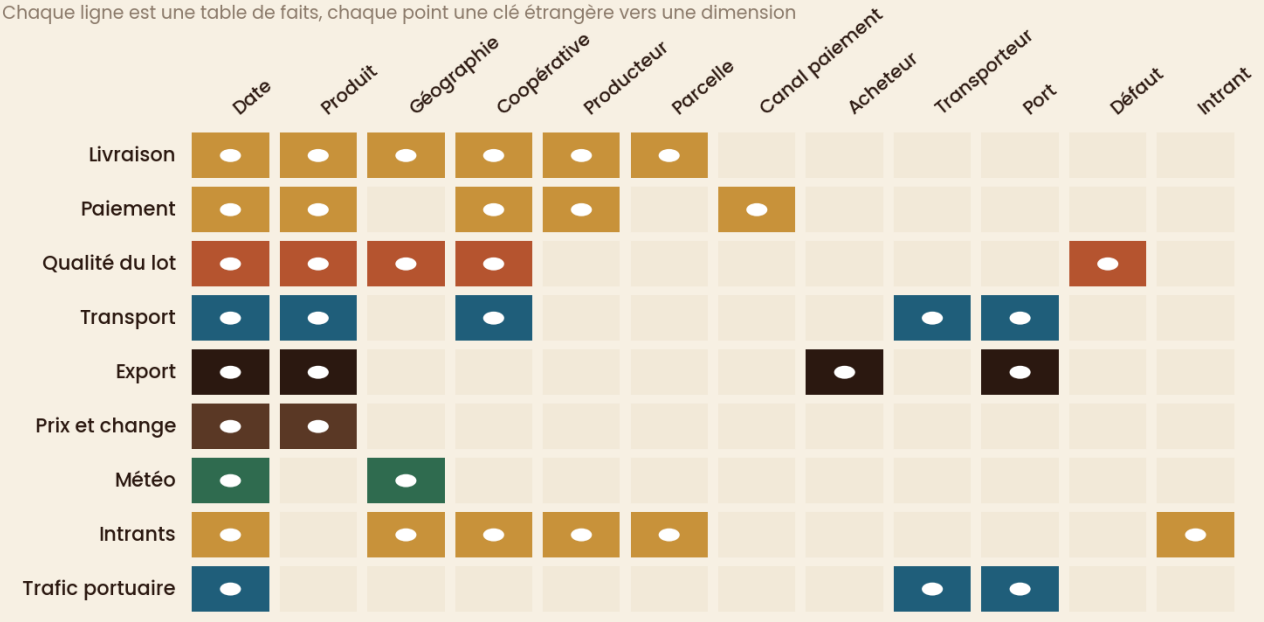
Des indicateurs aux processus, des processus aux faits, des faits au schéma en constellation.

La méthode en cinq étapes

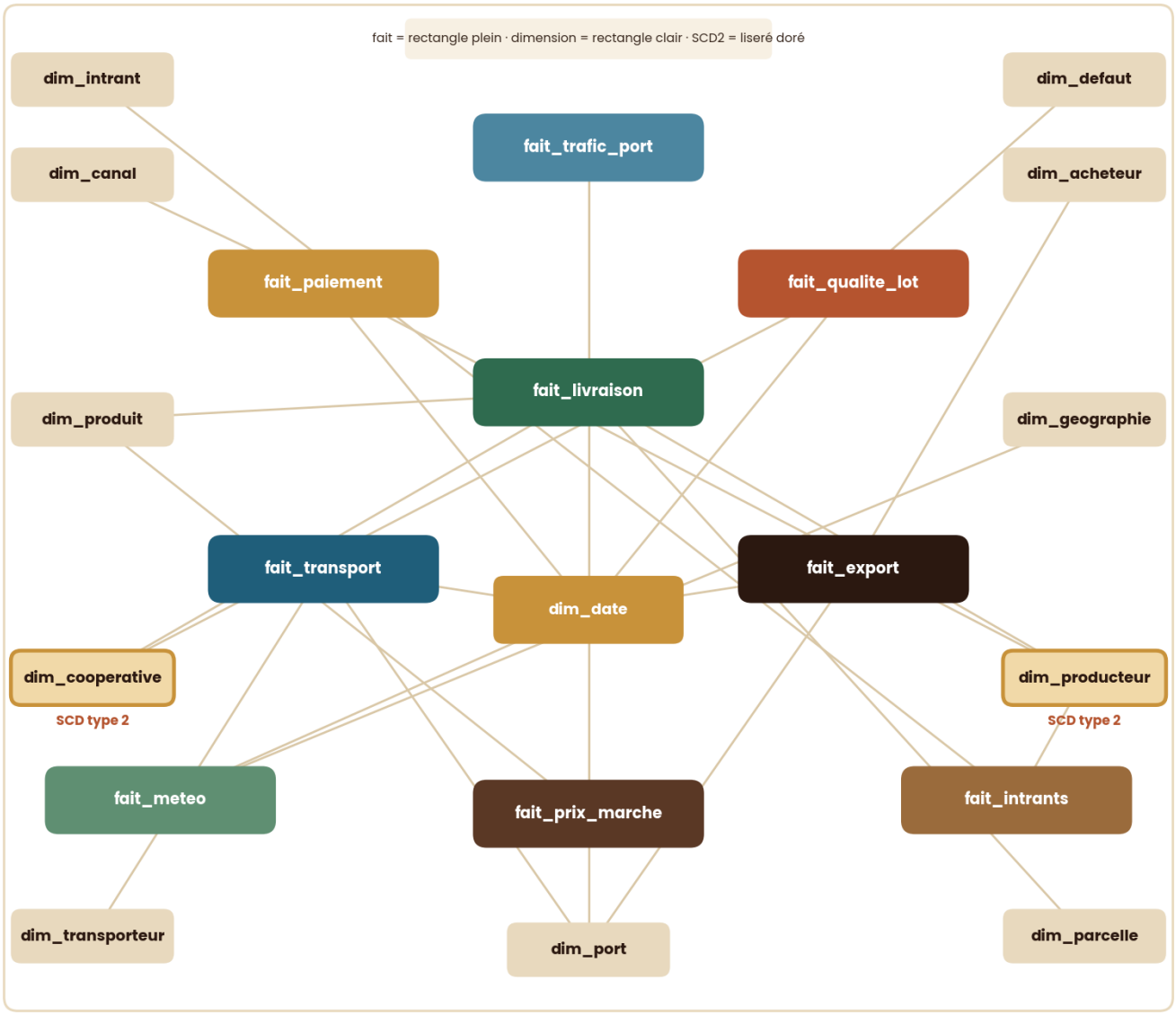
Étape	Action	Livrable
1	Partir des 25 indicateurs et des 6 questions de la direction	Tableau des indicateurs (chapitre 8)
2	Identifier les processus métier qui produisent les mesures : livraison, paiement, qualité, transport, export, marché, météo, intrants, trafic portuaire	Liste des processus
3	Déclarer la granularité de chaque fait, en une phrase, avant de dessiner une colonne	Tableau des grains
4	Choisir des dimensions conformes (partagées), dont une dimension de date commune	Matrice processus × dimensions
5	Lister les mesures et vérifier leur additivité ; décider quelles dimensions suivent l'historique (SCD2)	Schéma final et scripts DDL

Matrice processus × dimensions

Chaque ligne est une table de faits, chaque point une clé étrangère vers une dimension



Matrice des processus et des dimensions : le point de départ du schéma.



Constellation de 9 faits partageant 12 dimensions conformes. Les liserés dorés marquent les dimensions SCD de type 2.

Granularité justifiée

Le cahier des charges demande de justifier chaque grain. Voici l'argumentaire à reprendre dans votre dossier de conception.

Fait	Grain	Justification
fait_livraison	Une livraison d'un producteur à une coopérative	Plus fin que le lot : permet le tonnage par producteur, parcelle et jour, et se cumule à n'importe quel niveau
fait_paiement	Un paiement unitaire (rattaché à une livraison)	Le délai de paiement et le canal ne se calculent qu'à ce niveau. Statut conservé : seuls les paiements réussis comptent
fait_qualite_lot	Une analyse de lot	L'humidité et le taux de défauts sont des mesures du lot, pas de la livraison. Texte d'observation conservé pour le NLP
fait_transport	Un trajet camion de la coopérative au port	Les 600 000 pings GPS sont agrégés en trajets (environ 25 000) : durée, distance et pertes se lisent à ce niveau, sans alourdir le BI
fait_export	Une ligne de contrat expédiée	Volume, prix et montant sont portés par la ligne de contrat ; le nombre de conteneurs en découle
fait_prix_marche	Un prix par jour, produit et source	Les sources de prix ont des fréquences différentes : le jour est le plus fin commun, les agrégats mensuels se déduisent
fait_meteo	Un jour et un site climatique	Alimente les prévisions ; jointure à la région par dim_geographie

Fait	Grain	Justification
fait_intrants	Une distribution d'intrant à un producteur	Permet de relier intrants, rendement et coûts de la marge
fait_trafic_port	Un mouvement de conteneur ou une escale	Mesure l'attente à quai et les retards de chargement (indicateur 25)

Dimensions et historisation

Dimension	Clé naturelle	Attributs principaux	Historique
dim_date	date	année, mois, trimestre, jour de semaine, campagne (octobre à septembre)	Aucun
dim_produit	code_produit	CACAO, CAFE, variété (arabica, robusta), famille	Type 1
dim_geographie	code_localite	localité, département, région, latitude, longitude, station météo	Type 1
dim_cooperative	cooperative_bk	nom, région, capacité de stockage, responsable, certification	**SCD type 2**
dim_producteur	producteur_bk	nom, sexe, année de naissance, coopérative d'affiliation, certification, statut	**SCD type 2**
dim_parcelle	parcelle_bk	producteur, culture, surface (ha), année de plantation, géométrie	Type 1
dim_canal	code_canal	MTN MoMo, Orange Money, virement, espèces	Type 1
dim_acheteur	acheteur_bk	raison sociale, pays, type (négociant, torréfacteur)	Type 1
dim_transporteur	transporteur_bk	société, camion, capacité en tonnes	Type 1
dim_port	code_port	Douala, Kribi, terminal, pays	Type 1
dim_defaut	code_defaut	moisissure, grains ardoisés, infestation, humidité élevée, odeur de fumée, corps étrangers	Type 1
dim_intrant	code_intrant	famille, nom, unité	Type 1

Pourquoi le producteur et la coopérative en SCD2 ?
Un producteur change de coopérative ou obtient une certification : le tonnage d'avant doit rester attribué à son ancienne coopérative. En SCD2, chaque livraison pointe vers la version du producteur valide à sa date, grâce à une jointure sur l'intervalle date_debut, date_fin. C'est le comportement que vous démontrerez au run 2.

SQL · dwh.dim_producteur (SCD type 2) et clé unique de la version courante

```sql
CREATE TABLE dwh.dim_producteur (
    producteur_sk   BIGSERIAL PRIMARY KEY,                -- clé de substitution
    producteur_bk   VARCHAR(20)  NOT NULL,                -- clé naturelle de la source
    nom             VARCHAR(120),
    sexe            CHAR(1),
    annee_naissance SMALLINT,
    cooperative_bk  VARCHAR(20),
    certification   VARCHAR(30),
    statut          VARCHAR(15)  NOT NULL DEFAULT 'ACTIF',
    hash_diff       CHAR(32)     NOT NULL,                -- md5 des attributs suivis
    date_debut      DATE         NOT NULL,
    date_fin        DATE         NOT NULL DEFAULT '9999-12-31',
    est_courant     BOOLEAN      NOT NULL DEFAULT TRUE
);
CREATE UNIQUE INDEX ux_dim_producteur_courant ON dwh.dim_producteur (producteur_bk) WHERE est_courant;
```

SQL · dwh.fait_livraison (clés de substitution, mesures, traçabilité)

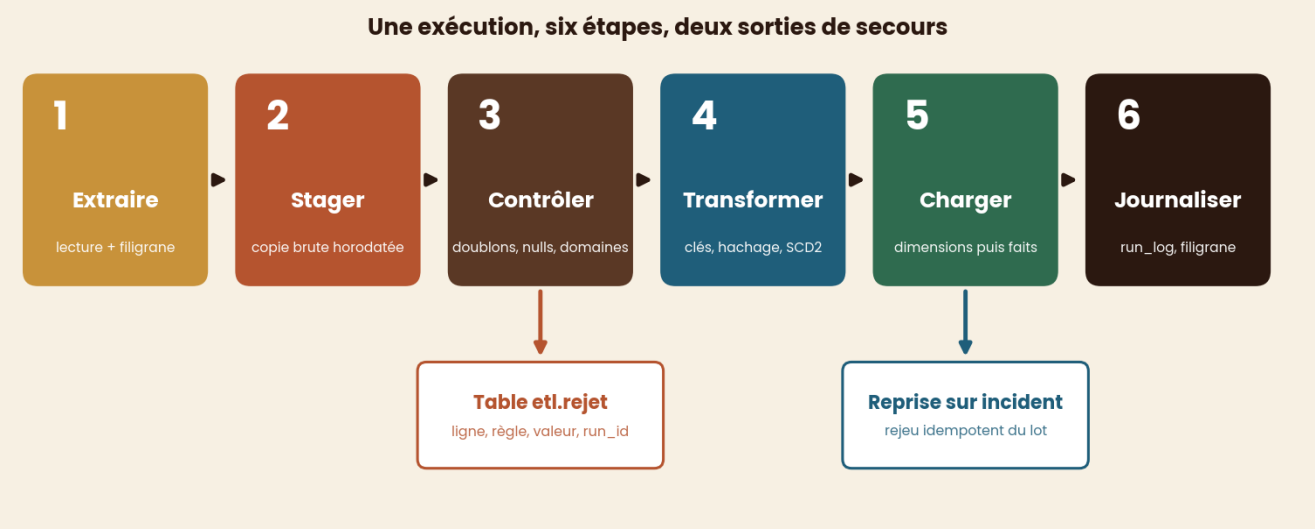
```sql
CREATE TABLE dwh.fait_livraison (
    livraison_sk   BIGSERIAL PRIMARY KEY,
    livraison_bk   VARCHAR(30) NOT NULL UNIQUE,            -- permet l'upsert idempotent
    date_sk        INT    NOT NULL REFERENCES dwh.dim_date,
    produit_sk     INT    NOT NULL REFERENCES dwh.dim_produit,
    cooperative_sk BIGINT NOT NULL REFERENCES dwh.dim_cooperative,
    producteur_sk  BIGINT NOT NULL REFERENCES dwh.dim_producteur,
    parcelle_sk    BIGINT REFERENCES dwh.dim_parcelle,
    geo_sk         INT    REFERENCES dwh.dim_geographie,
    poids_brut_kg  NUMERIC(10,2),
    poids_net_kg   NUMERIC(10,2),
    nb_sacs        INT,
    est_supprime   BOOLEAN NOT NULL DEFAULT FALSE,         -- suppression logique
    etl_run_id     BIGINT  NOT NULL,                       -- lien vers etl.run_log
    maj_le         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_livraison_date ON dwh.fait_livraison (date_sk);
CREATE INDEX ix_livraison_coop ON dwh.fait_livraison (cooperative_sk);
```

SQL · dwh.dim_date (générée une fois, de 2000 à 2030, avec la campagne octobre-septembre)

```sql
INSERT INTO dwh.dim_date (date_sk, date_complete, annee, mois, nom_mois, trimestre, jour_semaine, campagne, est_weekend)
SELECT to_char(d, 'YYYYMMDD')::int, d::date, EXTRACT(year FROM d), EXTRACT(month FROM d), to_char(d, 'TMMonth'),
       EXTRACT(quarter FROM d), EXTRACT(isodow FROM d),
       CASE WHEN EXTRACT(month FROM d) >= 10
            THEN EXTRACT(year FROM d) || '/' || (EXTRACT(year FROM d) + 1)
            ELSE (EXTRACT(year FROM d) - 1) || '/' || EXTRACT(year FROM d) END,
       EXTRACT(isodow FROM d) >= 6
FROM generate_series('2000-01-01'::date, '2030-12-31'::date, interval '1 day') AS g(d);
```

06 Construire l'ETL

Six étapes à chaque exécution, des contrôles qui rejettent proprement, et un journal qui permet de tout rejouer.



Le cycle d'une exécution et ses deux sorties de secours : table des rejets et reprise.

Ce que fait chaque étape

Étape	Actions	Trace laissée
1 · Extraire	Lire la source à partir du filigrane (SQL, fichier nouveau, API sur la fenêtre). Appel d'API avec délai d'attente et repli sur le cache	Ligne run_log en statut RUNNING
2 · Stager	Copier les données telles quelles dans stg.*, avec run_id et charge_le. Aucune transformation à ce stade	Nombre de lignes lues
3 · Contrôler	Appliquer les 10 règles de qualité. Les lignes en défaut vont dans etl.rejet avec la règle, la clé et la valeur	Lignes rejetées par règle
4 · Transformer	Standardiser (dates, unités, casse), dériver (prix en FCFA/kg), résoudre les clés de substitution, calculer les hachages	—
5 · Charger	Dimensions d'abord (SCD2 compris), faits ensuite, en transaction. Upsert sur la clé métier	Insérées, mises à jour, supprimées
6 · Journaliser	Fermer la ligne run_log, avancer le filigrane (même transaction que le chargement), déplacer les fichiers vers archive ou quarantaine	Statut SUCCESS ou FAILED

Les 10 règles de qualité

Règle	Contrôle	Action en cas d'échec
R01	Doublon sur la clé métier (livraison_id, paiement_id, lot_id)	Garder la ligne la plus récente, rejeter les autres
R02	Champs obligatoires : producteur_id, date, poids_net_kg non nuls	Rejeter la ligne
R03	Domaines de valeurs : 0 ≤ humidité ≤ 100, poids_net_kg > 0, poids_net ≤ poids_brut	Rejeter la ligne ; alerte si humidité supérieure à 15 %
R04	Intégrité référentielle : producteur, coopérative et livraison existent	Rejeter, ou rattacher au membre « Inconnu » selon la table
R05	Dates valides, formats normalisés vers ISO 8601	Corriger si possible, sinon rejeter
R06	Paiement : montant strictement positif, statut REUSSI pour être compté	Conserver en staging, exclure des indicateurs
R07	Cohérence temporelle : date_paiement ≥ date_livraison, date_analyse ≥ date_reception	Rejeter la ligne
R08	GPS : coordonnées dans la boîte englobante du pays, vitesse ≤ 120 km/h	Rejeter le ping
R09	Fichier XML conforme à son schéma XSD	Mettre le fichier en quarantaine
R10	Texte libre vide ou illisible (observation, clause)	Remplacer par NON_RENSEIGNE, comptabiliser

Tables techniques

Trois tables suffisent à tout piloter : le journal des exécutions, le filigrane par source et les rejets. Elles servent aussi pour l'indicateur « date du dernier chargement » exigé dans le tableau de bord.

SQL · schéma etl : journalisation, filigranes, rejets, fichiers traités

```sql
CREATE TABLE etl.run_log (
    run_id            BIGSERIAL PRIMARY KEY,
    dag_id            TEXT, task_id TEXT, source_name TEXT NOT NULL,
    mode              VARCHAR(8)  NOT NULL CHECK (mode IN ('FULL','INCR')),
    declencheur       VARCHAR(12) CHECK (declencheur IN ('PLANIFIE','FICHIER','EVENEMENT','MANUEL')),
    debut             TIMESTAMPTZ NOT NULL DEFAULT now(),
    fin               TIMESTAMPTZ,
    statut            VARCHAR(10) NOT NULL DEFAULT 'RUNNING' CHECK (statut IN ('RUNNING','SUCCESS','FAILED')),
    lignes_lues INT, lignes_inserees INT, lignes_maj INT, lignes_supprimees INT, lignes_rejetees INT,
    watermark_avant   TEXT, watermark_apres TEXT,
    message_erreur    TEXT
);
CREATE TABLE etl.watermark (source_name TEXT PRIMARY KEY, derniere_valeur TEXT NOT NULL, maj_le TIMESTAMPTZ DEFAULT now());
CREATE TABLE etl.rejet (
    rejet_id BIGSERIAL PRIMARY KEY, run_id BIGINT REFERENCES etl.run_log, source_name TEXT,
    cle_metier TEXT, regle VARCHAR(4), valeur_rejetee JSONB, cree_le TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE etl.fichier_traite (nom TEXT PRIMARY KEY, hash_md5 CHAR(32), run_id BIGINT, traite_le TIMESTAMPTZ DEFAULT now());
```

Reprise sur incident : cinq règles

- **Tout charger dans une transaction par tâche** : si la tâche échoue, rien n'est à moitié écrit.
- **Avancer le filigrane dans la même transaction** que le chargement. Après un échec, le filigrane n'a pas bougé : on relit exactement les mêmes lignes.
- **Écrire des opérations idempotentes** : upsert sur la clé métier (ON CONFLICT), staging purgé par run_id avant relecture. Rejouer deux fois ne crée aucun doublon.
- **Trier les fichiers** : succès vers data/archive, échec ou XML non valide vers data/quarantaine, avec une ligne dans etl.fichier_traite.
- **Alerter** : Airflow relance deux fois (retries) puis envoie un message ; le statut FAILED reste visible dans le journal.

Le geste de démonstration à répéter
Au run 4, faites échouer volontairement la tâche de chargement des faits (une exception levée au milieu). Montrez le statut FAILED dans run_log, le filigrane inchangé, puis relancez : statut SUCCESS, nombre de lignes correct, aucun doublon. C'est la preuve la plus convaincante de la reprise sur incident.

07 Incrémental et déclenchement automatique

Ne lire que le nouveau, détecter les modifications et les suppressions, et lancer le tout sans intervention humaine.

Quelle stratégie pour quelle source ?

Source	Nouvelles lignes	Lignes modifiées	Lignes supprimées
Livraisons (PostgreSQL)	Filigrane updated_at, avec 5 minutes de recouvrement	Même filigrane : upsert sur livraison_id	Journal coop.cdc_journal (opération D) ou comparaison des clés
Paiements (JSON)	Un fichier par jour ; table etl.fichier_traite	Hachage du fichier : fichier corrigé, relu	Statut ANNULE dans le fichier suivant
Contrôle qualité (JSON)	Filigrane date_analyse	Hachage de ligne (md5 des colonnes suivies)	Non applicable : décision conservée
Parcelles et producteurs (GeoJSON)	Instantané complet comparé à la dimension	hash_diff par producteur : nouvelle version SCD2	Absence dans l'instantané : statut RADIE (nouvelle version)
Transport (logs)	Décalage lu : dernier horodatage chargé	Journal en ajout seul : aucune	Aucune
Contrats (XML)	Identifiant de contrat inconnu	Hachage du fichier : avenant = nouvelle version	Contrat annulé : balise statut
Intrants, trafic portuaire	Hachage du fichier, identifiant d'escale ou de ligne	Hachage de ligne	Lignes absentes d'un classeur remplacé
Prix et FAOSTAT	Clé (date, série) inconnue	Hachage de ligne : valeurs révisées	Non applicable
NASA POWER, Open-Meteo, change	Dernier jour chargé plus un	Fenêtre glissante de 7 jours : valeurs recalculées	Non applicable

SQL · détection des modifications par hachage et SCD2 en deux temps (une seule transaction)

```sql
-- hash_diff calculé en staging : md5 des seuls attributs que l'on veut historiser
UPDATE stg.producteur SET hash_diff = md5(concat_ws('|', nom, cooperative_id, certification, statut));

-- date_effet = propriété maj_le du GeoJSON (2000-01-01 au chargement initial)
-- 1) fermer la version courante des producteurs dont les attributs ont changé
UPDATE dwh.dim_producteur d
   SET date_fin = s.date_effet - 1, est_courant = FALSE
  FROM stg.producteur s
 WHERE d.producteur_bk = s.producteur_id AND d.est_courant AND d.hash_diff <> s.hash_diff;

-- 2) insérer les nouveaux producteurs ET les nouvelles versions (plus de version courante pour eux)
INSERT INTO dwh.dim_producteur (producteur_bk, nom, sexe, annee_naissance, cooperative_bk, certification, statut, hash_diff, date_debut)
SELECT s.producteur_id, s.nom, s.sexe, s.annee_naissance, s.cooperative_id, s.certification, s.statut, s.hash_diff, s.date_effet
  FROM stg.producteur s
  LEFT JOIN dwh.dim_producteur d ON d.producteur_bk = s.producteur_id AND d.est_courant
 WHERE d.producteur_bk IS NULL;
```

SQL · chargement incrémental de fait_livraison : upsert idempotent avec jointure temporelle sur le SCD2

```sql
INSERT INTO dwh.fait_livraison (livraison_bk, date_sk, produit_sk, cooperative_sk, producteur_sk, parcelle_sk,
                                poids_brut_kg, poids_net_kg, nb_sacs, etl_run_id)
SELECT s.livraison_id, dd.date_sk, pr.produit_sk, c.cooperative_sk, p.producteur_sk, pa.parcelle_sk,
       s.poids_brut_kg, s.poids_net_kg, s.nb_sacs, :run_id
  FROM stg.livraison_propre s
  JOIN dwh.dim_date        dd ON dd.date_complete = s.date_livraison::date
  JOIN dwh.dim_produit     pr ON pr.code_produit  = s.produit
  JOIN dwh.dim_cooperative c  ON c.cooperative_bk = s.cooperative_id
                             AND s.date_livraison::date BETWEEN c.date_debut AND c.date_fin
  JOIN dwh.dim_producteur  p  ON p.producteur_bk  = s.producteur_id
                             AND s.date_livraison::date BETWEEN p.date_debut AND p.date_fin
  LEFT JOIN dwh.dim_parcelle pa ON pa.parcelle_bk = s.parcelle_id
ON CONFLICT (livraison_bk) DO UPDATE
   SET poids_brut_kg = EXCLUDED.poids_brut_kg, poids_net_kg = EXCLUDED.poids_net_kg, nb_sacs = EXCLUDED.nb_sacs,
       est_supprime = FALSE, etl_run_id = EXCLUDED.etl_run_id, maj_le = now();

-- suppressions : clés présentes dans l'entrepôt mais absentes de l'instantané des clés de la source
UPDATE dwh.fait_livraison f SET est_supprime = TRUE, maj_le = now(), etl_run_id = :run_id
 WHERE NOT f.est_supprime
   AND NOT EXISTS (SELECT 1 FROM stg.livraison_cles k WHERE k.livraison_id = f.livraison_bk);
```

Dates d'effet du SCD2

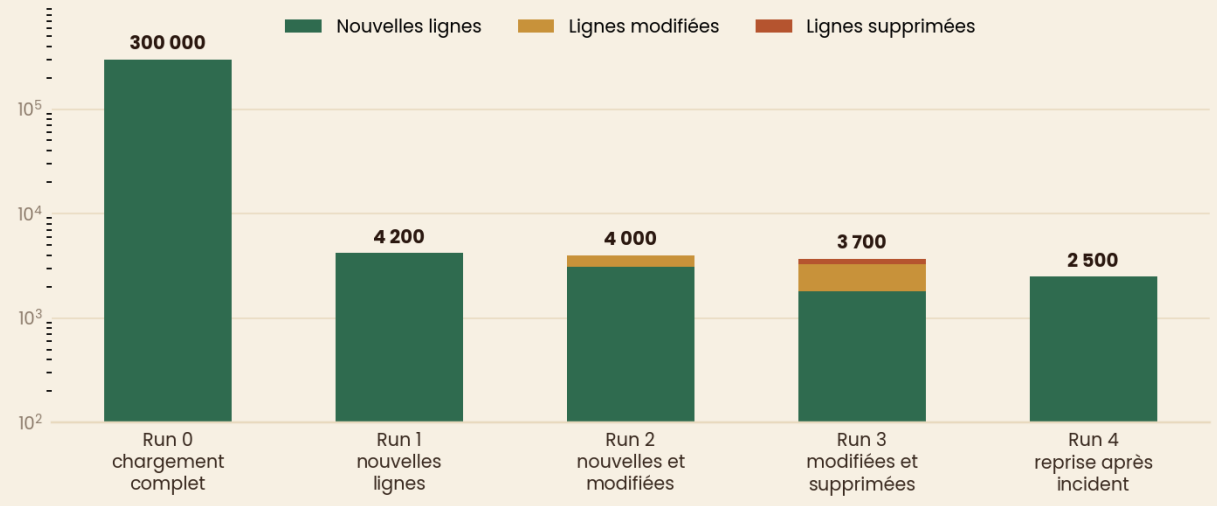
Utilisez la date d'effet fournie par la source (propriété maj_le du GeoJSON) et non la date du jour : ainsi les livraisons postérieures au changement de coopérative pointent bien vers la nouvelle version, et les anciennes vers l'ancienne. Au chargement initial, fixez date_effet à 2000-01-01 pour que tout l'historique se rattache à la première version.

Attention aux suppressions

Les indicateurs doivent toujours filtrer sur est_supprime = FALSE. Gardez la ligne (suppression logique) plutôt que de la détruire : cela préserve l'historique et permet d'expliquer pourquoi un tonnage a baissé entre deux chargements.

Scénario de démonstration du chargement incrémental (table fait_livraison)

Volumes indicatifs, échelle logarithmique



Les cinq runs de la démonstration : un chargement complet, puis quatre chargements incrémentaux.

Script de démonstration et résultats attendus

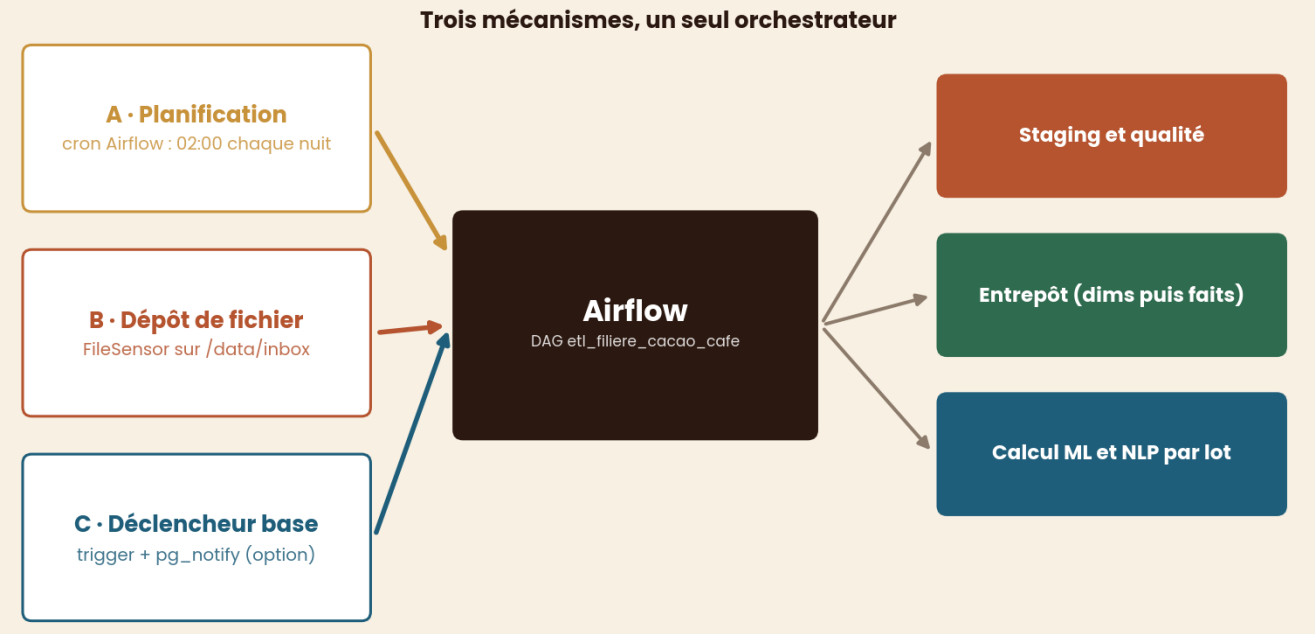
Run	Action sur la source	Résultat attendu dans l'entrepôt
0	Chargement complet initial (300 000 livraisons, 25 000 producteurs)	fait_livraison = 300 000 ; dim_producteur = 25 000 lignes toutes courantes
1	Insertion de 4 200 livraisons nouvelles	304 200 livraisons ; aucune version de dimension créée
2	3 100 nouvelles, 900 corrections de poids, 120 producteurs changent de coopérative	307 300 livraisons ; 900 poids mis à jour ; dim_producteur = 25 120 lignes dont 25 000 courantes
3	1 800 nouvelles, 1 500 modifiées, 400 supprimées	309 100 livraisons dont 400 est_supprime ; tonnage actif en baisse expliquée
4	2 500 nouvelles, mais la tâche de chargement des faits échoue au milieu	FAILED, filigrane inchangé ; après relance : 311 600 livraisons, zéro doublon

SQL · la requête à projeter pendant la démonstration

```sql
SELECT run_id, source_name, mode, declencheur, statut, lignes_lues, lignes_inserees, lignes_maj,
       lignes_supprimees, lignes_rejetees, watermark_avant, watermark_apres, round(extract(epoch FROM fin - debut)) AS secondes
  FROM etl.run_log
 WHERE source_name = 'livraison'
 ORDER BY run_id DESC
 LIMIT 10;
```

Déclenchement automatique : trois mécanismes

Le cahier des charges en exige deux au moins. Nous recommandons de réaliser A et B pour sécuriser la note, et C si le temps le permet : il impressionne parce qu'il réagit à un événement de la base.



Trois portes d'entrée, un seul orchestrateur, les mêmes tâches derrière.

Mécanisme	Comment	Comment le démontrer
A · Planification	DAG nocturne (cron 0 2 * * *) : staging, qualité, chargement, calcul ML et NLP	Montrer le planning dans Airflow et un run déclenché par l'horloge (colonne declencheur = PLANIFIE)
B · Dépôt de fichier	Second DAG toutes les 10 minutes avec un capteur de fichier sur data/inbox. Dès qu'un paiements_AAAAMMJJ.json arrive, il est traité et archivé	Copier un fichier dans inbox pendant la soutenance, voir le DAG démarrer seul, puis le fichier passer dans archive

Mécanisme	Comment	Comment le démontrer
C · Déclencheur de base (option)	Un déclencheur sur coop.livraison écrit dans cdc_journal et émet un pg_notify ; un petit écouteur Python appelle l'API d'Airflow	Insérer une livraison dans la source et voir l'exécution partir, avec declencheur = EVENEMENT

Python · dags/etl_filiere_cacao_cafe.py (mécanisme A : planification nocturne)

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from etl.pipeline import extraire, controler, charger_dimensions, charger_faits, calculer_ml_nlp

with DAG(
    dag_id="etl_filiere_cacao_cafe",
    schedule="0 2 * * *",                    # tous les jours à 02:00
    start_date=datetime(2026, 1, 1), catchup=False, max_active_runs=1,
    default_args={"retries": 2, "retry_delay": timedelta(minutes=5)},
) as dag:
    t1 = PythonOperator(task_id="extraire_vers_staging", python_callable=extraire, op_kwargs={"declencheur": "PLANIFIE"})
    t2 = PythonOperator(task_id="controler_qualite", python_callable=controler)
    t3 = PythonOperator(task_id="charger_dimensions", python_callable=charger_dimensions)
    t4 = PythonOperator(task_id="charger_faits", python_callable=charger_faits)
    t5 = PythonOperator(task_id="calcul_ml_nlp", python_callable=calculer_ml_nlp)
    t1 >> t2 >> t3 >> t4 >> t5
```

Python · dags/etl_depot_fichier.py (mécanisme B : capteur sur le dossier de dépôt ; imports selon votre version d'Airflow)

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.sensors.filesystem import FileSensor
from etl.pipeline import traiter_paiements

with DAG(
    dag_id="etl_depot_fichier",
    schedule="*/10 * * * *", start_date=datetime(2026, 1, 1), catchup=False, max_active_runs=1,
) as dag:
    attendre = FileSensor(
        task_id="attendre_paiements", fs_conn_id="fs_default", filepath="/data/inbox/paiements_*.json",
        poke_interval=30, timeout=240, mode="reschedule", soft_fail=True,   # pas de fichier : exécution ignorée, pas en échec
    )
    traiter = PythonOperator(task_id="traiter_et_archiver", python_callable=traiter_paiements, op_kwargs={"declencheur": "FICHIER"})
    attendre >> traiter
```

SQL · mécanisme C : déclencheur de journalisation sur la table source (BEFORE pour pouvoir fixer updated_at)

```sql
CREATE TABLE coop.cdc_journal (cdc_id BIGSERIAL PRIMARY KEY, table_nom TEXT, operation CHAR(1), cle_metier TEXT, horodatage TIMESTAMPTZ

CREATE OR REPLACE FUNCTION coop.journaliser_livraison() RETURNS trigger AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    INSERT INTO coop.cdc_journal (table_nom, operation, cle_metier) VALUES ('livraison', 'D', OLD.livraison_id);
    PERFORM pg_notify('livraison_changee', OLD.livraison_id);
    RETURN OLD;
  END IF;
  NEW.updated_at := now();
  INSERT INTO coop.cdc_journal (table_nom, operation, cle_metier)
  VALUES ('livraison', CASE TG_OP WHEN 'INSERT' THEN 'I' ELSE 'U' END, NEW.livraison_id);
  PERFORM pg_notify('livraison_changee', NEW.livraison_id);
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_livraison BEFORE INSERT OR UPDATE OR DELETE ON coop.livraison
FOR EACH ROW EXECUTE FUNCTION coop.journaliser_livraison();
```

08 Les 25 indicateurs

Cinq pages, cinq indicateurs chacune : définition, formule et table de faits qui les porte.

Chaque indicateur est défini une seule fois, ici, puis réutilisé partout : requête SQL, carte du tableau de bord, dossier de conception. Les définitions suivent le cahier des charges ; les formules sont les nôtres et se justifient en soutenance.

Page 1 · Synthèse

N°	Indicateur	Définition et formule	Faits
1	**Tonnage collecté**	Somme des poids nets livrés, en tonnes (poids_net_kg / 1000), hors lignes supprimées ; par campagne et produit	fait_livraison
2	**Prix moyen payé au producteur**	Somme des montants payés (statut réussi) divisée par la somme des kilos livrés, en FCFA/kg : moyenne pondérée par le poids	fait_paiement, fait_livraison
3	**Taux de lots refusés**	Lots refusés divisés par lots analysés, en pourcentage	fait_qualite_lot
4	**Marge de la filière**	(CA export - achats producteurs - intrants - transport - frais portuaires) divisé par le CA export	fait_export, fait_paiement, fait_intrants, fait_transport
5	**Chiffre d'affaires à l'exportation**	Somme des volumes expédiés (t) × prix du contrat, converti en FCFA au taux du jour de facturation	fait_export, fait_prix_marche

Page 2 · Production et collecte

N°	Indicateur	Définition et formule	Faits
6	**Tonnage par région**	Indicateur 1 ventilé par région de la dimension géographique	fait_livraison
7	**Tonnage par coopérative**	Indicateur 1 ventilé par coopérative (version valide à la date de livraison)	fait_livraison
8	**Rendement par hectare**	Tonnage de la campagne divisé par la surface cultivée des parcelles actives, en t/ha	fait_livraison, dim_parcelle
9	**Nombre de producteurs actifs**	Producteurs distincts ayant au moins une livraison sur la période (12 mois glissants pour la valeur de référence)	fait_livraison
10	**Nombre de parcelles actives**	Parcelles distinctes rattachées à au moins une livraison de la période	fait_livraison

Page 3 · Qualité

N°	Indicateur	Définition et formule	Faits
11	**Taux d'humidité moyen**	Moyenne de humidite_pct pondérée par le poids du lot ; seuil de référence paramétrable (par exemple 8 %)	fait_qualite_lot
12	**Taux de grains défectueux**	Grains défectueux divisés par grains comptés à l'épreuve de coupe, en pourcentage	fait_qualite_lot
13	**Part des lots de grade 1**	Lots de grade 1 divisés par lots acceptés	fait_qualite_lot
14	**Lots refusés par coopérative**	Nombre et taux de lots refusés, par coopérative	fait_qualite_lot
15	**Délai d'analyse qualité**	Moyenne de (date_analyse - date_reception) en heures ou en jours	fait_qualite_lot

Page 4 · Prix et finances

N°	Indicateur	Définition et formule	Faits
16	**Écart entre prix producteur et prix mondial**	Prix mondial converti en FCFA/kg moins prix producteur, et part du prix mondial perçue par le producteur	fait_paiement, fait_prix_marche
17	**Évolution mensuelle du prix**	Variation en pourcentage d'un mois sur l'autre du prix mondial et du prix producteur	fait_prix_marche, fait_paiement
18	**Montant payé via Mobile Money**	Somme des paiements réussis dont le canal est MTN MoMo ou Orange Money	fait_paiement
19	**Délai moyen de paiement**	Moyenne de (date_paiement - date_livraison), en jours, sur les paiements réussis	fait_paiement
20	**Impact du taux de change**	Écart de CA export entre le taux réel et un taux de référence : somme des CA en USD × (taux réel - taux de référence)	fait_export, fait_prix_marche

Page 5 · Logistique et export

N°	Indicateur	Définition et formule	Faits
21	**Délai champ vers port**	Date d'arrivée au port moins date de livraison à la coopérative, en jours (moyenne)	fait_transport, fait_livraison
22	**Coût de transport par tonne**	Somme des coûts de transport divisée par les tonnes transportées	fait_transport
23	**Taux de pertes en transport**	(poids au départ - poids à l'arrivée) divisé par le poids au départ	fait_transport
24	**Conteneurs exportés**	Nombre de conteneurs (EVP) chargés et expédiés	fait_export
25	**Retards de chargement**	Part des chargements dont la date réelle dépasse la date prévue, et retard moyen en heures	fait_trafic_port

SQL · indicateur 2 : prix moyen payé au producteur (moyenne pondérée par le poids)

```sql
SELECT d.campagne,
       SUM(p.montant_fcfa) / NULLIF(SUM(l.poids_net_kg), 0) AS prix_moyen_fcfa_kg
  FROM dwh.fait_paiement p
  JOIN dwh.fait_livraison l ON l.livraison_sk = p.livraison_sk
  JOIN dwh.dim_date d       ON d.date_sk = l.date_sk
 WHERE p.statut = 'REUSSI' AND NOT l.est_supprime
 GROUP BY d.campagne
 ORDER BY d.campagne;
```

SQL · indicateur 3 : taux de lots refusés, et date du dernier chargement pour l'en-tête de chaque page

```sql
SELECT d.campagne, round(100.0 * SUM(q.est_refuse::int) / COUNT(*), 2) AS taux_refus_pct
  FROM dwh.fait_qualite_lot q JOIN dwh.dim_date d ON d.date_sk = q.date_analyse_sk
 GROUP BY d.campagne;

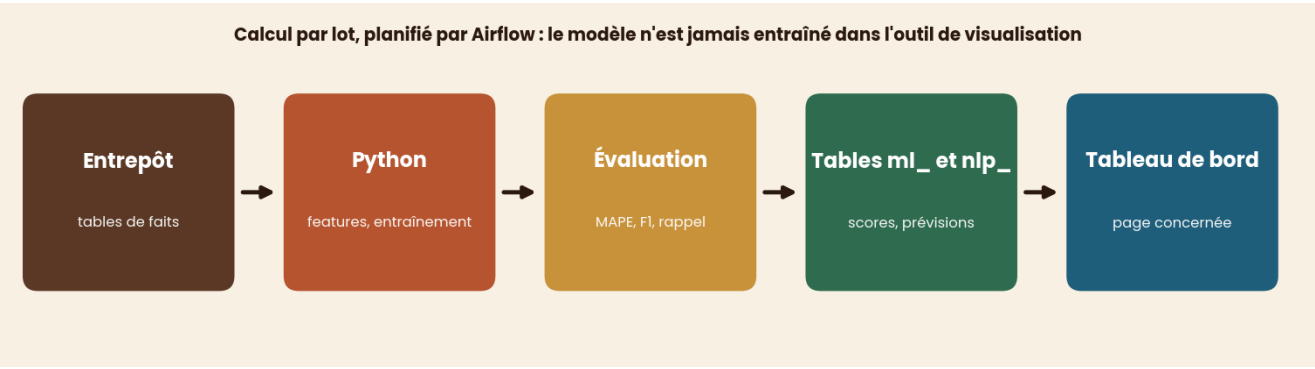
SELECT max(fin) AS dernier_chargement FROM etl.run_log WHERE statut = 'SUCCESS';
```

Deux pièges de calcul à éviter

Ne faites jamais la moyenne de moyennes : le prix moyen, l'humidité et les taux se calculent toujours par rapport de sommes (somme divisée par somme). Et ne mélangez pas les devises : tout montant de marché est converti en FCFA au taux de sa date avant d'être additionné.

09 Machine Learning et NLP

Trois fonctionnalités de ML, deux de NLP : entraînées hors de l'outil, calculées par lot, restituées avec leurs métriques.



Le circuit commun aux cinq fonctionnalités.

Le cahier des charges est explicite : les modèles sont entraînés en Python, leurs résultats sont écrits dans l'entrepôt par une tâche ETL planifiée, puis le tableau de bord les affiche avec leurs métriques d'évaluation. Chaque fonctionnalité ci-dessous précise ses données, sa méthode, sa mesure de qualité et sa table de sortie.

La règle d'or : un modèle honnête vaut mieux qu'un modèle spectaculaire

Comparez toujours à une référence simple (valeur de l'an dernier, règle seuil, mots-clés). Évaluez sur des données postérieures à l'entraînement, jamais mélangées au hasard. Un score de 100 % ou un modèle qui ne bat pas la référence sont deux signaux d'alerte à expliquer, pas à cacher.

ML 1	Prévision des tonnages par région et par campagne · page 2
Objectif	Anticiper les tonnages mensuels de chaque région pour la campagne en cours et la suivante, afin de planifier collecte, stockage et transport
Données	fait_livraison agrégé par mois, région et produit (4 campagnes) ; fait_meteo (pluie cumulée et température décalées de 1 à 3 mois) ; prix producteur décalé
Modèles	Référence : valeur du même mois un an plus tôt. Modèle : Prophet avec régresseurs météo, ou SARIMAX (statsmodels), ou gradient boosting avec retards
Évaluation	Validation à origine glissante (on entraîne jusqu'à un mois, on prédit le suivant, on avance). Métriques : MAPE, WAPE et gain par rapport à la référence
Sortie	ml.prevision_tonnage (region_sk, produit_sk, mois_cible, yhat, yhat_bas, yhat_haut, modele, run_id, calcule_le) et ml.metrique_modele
Précaution	Aucune donnée future dans les variables explicatives. Afficher l'intervalle d'incertitude, pas seulement la courbe centrale

ML 2	Prédiction du rejet d'un lot · page 3
Objectif	Estimer la probabilité qu'un lot soit refusé à partir des mesures de qualité, pour orienter les contrôles
Cible et variables	Cible : est_refuse. Variables : humidité, taux de grains défectueux, grains comptés, produit, région, mois, délai entre livraison et analyse. Ne jamais utiliser le grade, qui est une conséquence de la décision
Modèles	Référence : régression logistique. Modèle : forêt aléatoire ou gradient boosting. Rejet rare (environ 5 %) : poids de classes, seuil choisi sur la courbe précision-rappel
Évaluation	Découpage dans le temps (entraînement sur les anciennes campagnes, test sur la dernière). Précision, rappel et F1 de la classe « refusé », PR-AUC, matrice de confusion
Sortie	ml.score_rejet_lot (lot_sk, proba_rejet, classe_predite, seuil, modele, run_id)
Précaution	Générer le rejet de façon probabiliste à partir de l'humidité et des défauts (pas par règle exacte), sinon le score sera parfait ou nul

ML 3	Prévision du prix mondial et simulation de marge · page 4
Objectif	Prévoir le prix mondial à un à trois mois, puis mesurer la marge de la filière sous différents scénarios
Données	fait_prix_marche mensuel (Pink Sheet, ICCO, ICO) et taux de change ; coûts issus des paiements, intrants, transport et frais
Modèles	Référence : dernier prix observé. Modèles : ETS ou ARIMA, Prophet, gradient boosting avec retards. La simulation de marge est un calcul déterministe appliqué aux prévisions
Scénarios	Paramétrables : prix mondial ± 10 %, change ± 5 %, coût de transport + 15 %. Marge = CA export - (achats + intrants + transport + frais), exprimée en pourcentage du CA
Évaluation	MAE, RMSE et MAPE comparés à la référence sur validation glissante
Sortie	ml.prevision_prix (produit_sk, date_cible, yhat, yhat_bas, yhat_haut, modele, run_id) et ml.scenario_marge (scenario, parametres, marge_pct, run_id)
Honnêteté	Un prix de marché est très difficile à prévoir : un modèle qui ne bat pas la référence est un résultat acceptable s'il est mesuré et expliqué

NLP 1	Classification des observations de contrôle qualité par type de défaut · page 3
Objectif	Transformer les remarques libres des analystes (« lot humide, odeur de fumée, quelques grains moisis ») en types de défauts comptables
Données	Champ observation de 150 000 contrôles. Classes : moisissure, grains ardoisés, infestation, humidité élevée, odeur de fumée, corps étrangers, aucun défaut
Méthode	Nettoyage (minuscules, accents), TF-IDF sur mots et paires de mots, régression logistique ou LinearSVC. Option : CamemBERT (Hugging Face) pour aller plus loin
Jeu annoté	Chaque observation générée porte son étiquette réelle : environ 3 000 servent à l'entraînement, le reste à l'évaluation. Prévoir 40 à 60 gabarits de phrases par défaut, des fautes de frappe et 10 à 15 % de phrases multi-défauts
Évaluation	F1 macro, rappel par classe, matrice de confusion ; test sur des formulations absentes de l'entraînement
Sortie	nlp.defaut_observation (lot_sk, defaut_code, proba, modele, run_id) qui alimente dim_defaut et la page 3

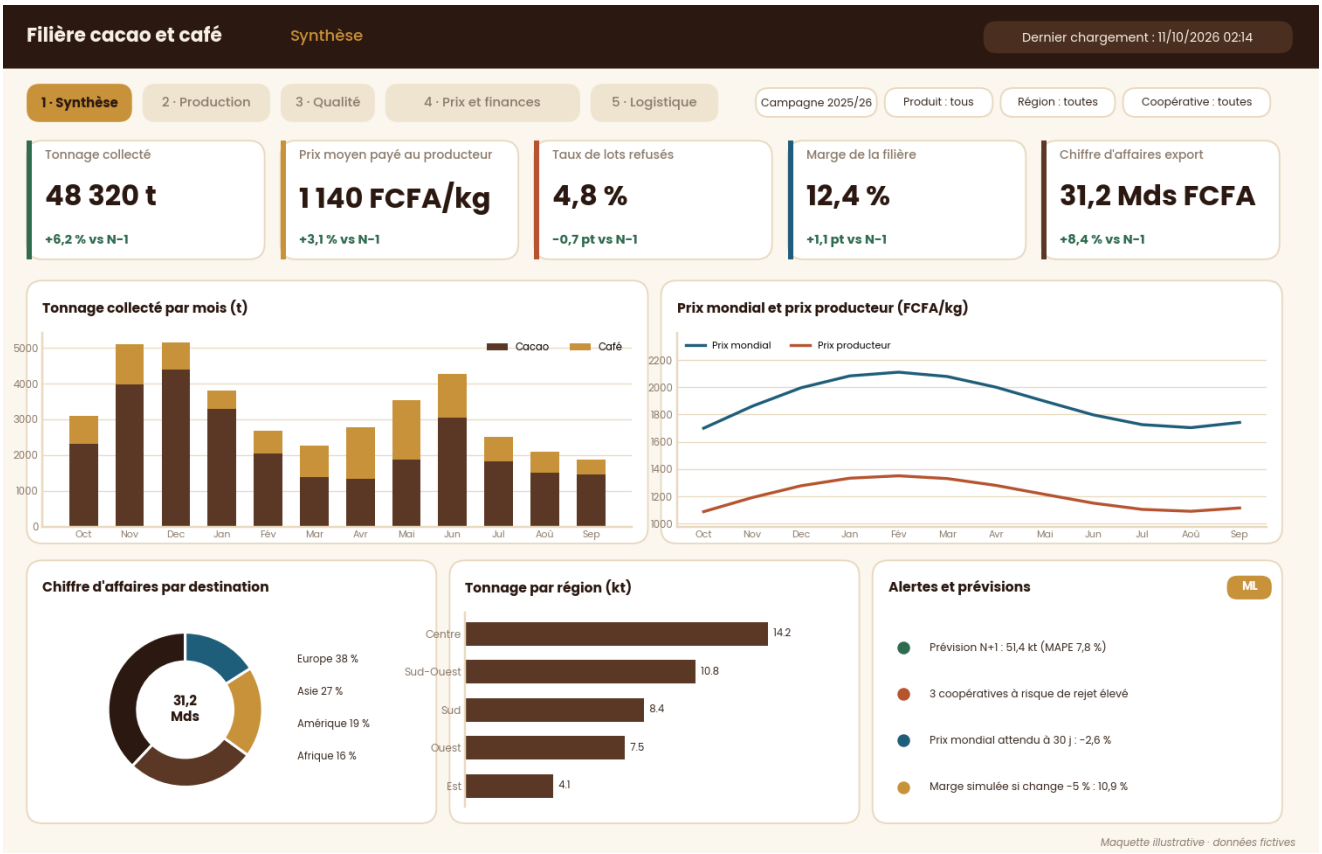
NLP 2	Extraction des clauses, dates et volumes des contrats d'exportation · page 5
Objectif	Lire le texte libre des clauses d'un contrat XML et en extraire les engagements : date limite d'expédition, volume, incoterm, port, pénalité de retard
Exemple de clause	« Expédition au plus tard le 15 novembre 2025, 250 tonnes FOB Douala, pénalité de 0,5 % par jour de retard. »
Méthode	spaCy : EntityRuler et Matcher pour les tonnes, incoterms, ports et pénalités ; normalisation des dates en ISO. Option : NER entraîné sur 100 à 150 clauses annotées
Vérité terrain	Le XML contient aussi les champs structurés (volume, port, incoterm) : ils servent à mesurer automatiquement la qualité de l'extraction
Évaluation	Précision, rappel et F1 par type d'entité, sur au moins 100 clauses n'ayant pas servi à écrire les règles ; formulations variées (« avant le », « au plus tard le », « 250 t »)
Sortie	nlp.contrat_extraction (contrat_sk, entite, valeur_texte, valeur_normalisee, confiance, run_id) qui alimente la page 5

10 Le tableau de bord en 5 pages

Une page principale, quatre pages métier,
des filtres communs, une navigation claire et la fraîcheur des données.

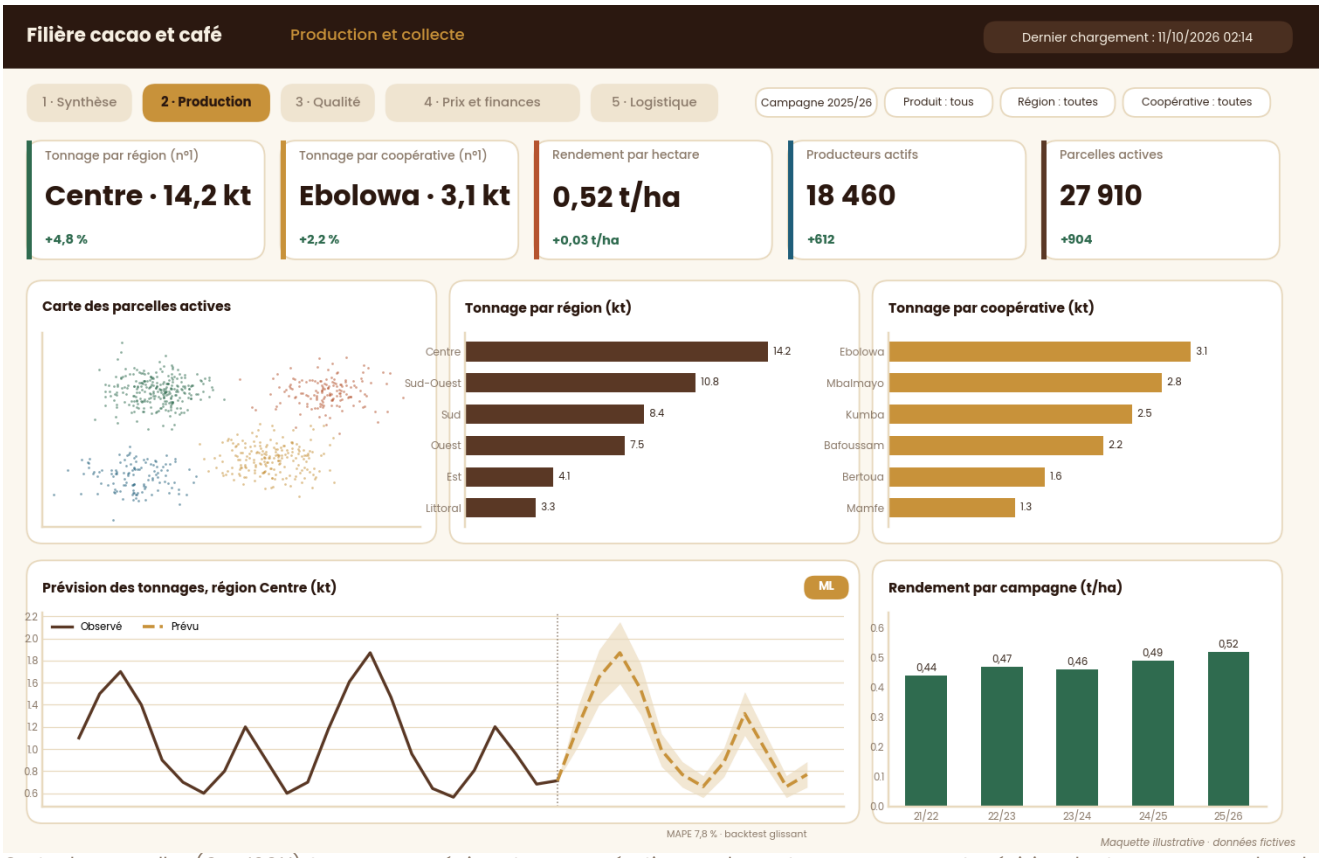
Les cinq maquettes suivantes montrent la composition cible. Elles sont illustratives (données fictives) ; adaptez-les à l'outil choisi. Chaque page a le même gabarit : bandeau de titre avec la date du dernier chargement, onglets de navigation, filtres communs, cinq cartes d'indicateurs, puis les graphiques et les apports du ML et du NLP.

Page 1 · Synthèse



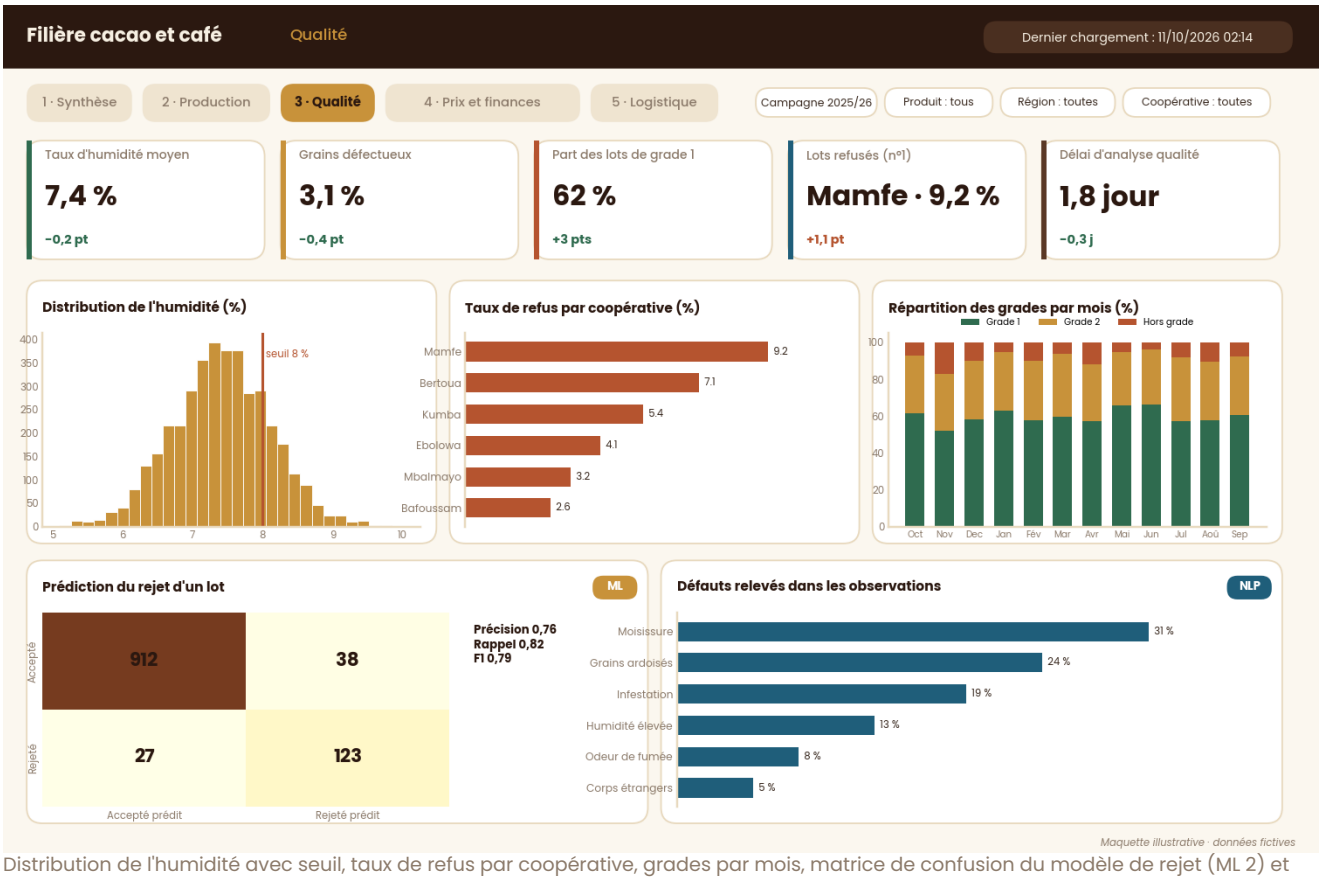
Les cinq indicateurs clés en cartes, la tendance du tonnage, le prix mondial face au prix producteur, la répartition du chiffre d'affaires, et un panneau d'alertes alimenté par les modèles.

Page 2 · Production et collecte



Carte des parcelles (GeoJSON), tonnage par région et par coopérative, rendement par campagne et prévision des tonnages avec bande d'incertitude (ML 1).

Page 3 · Qualité



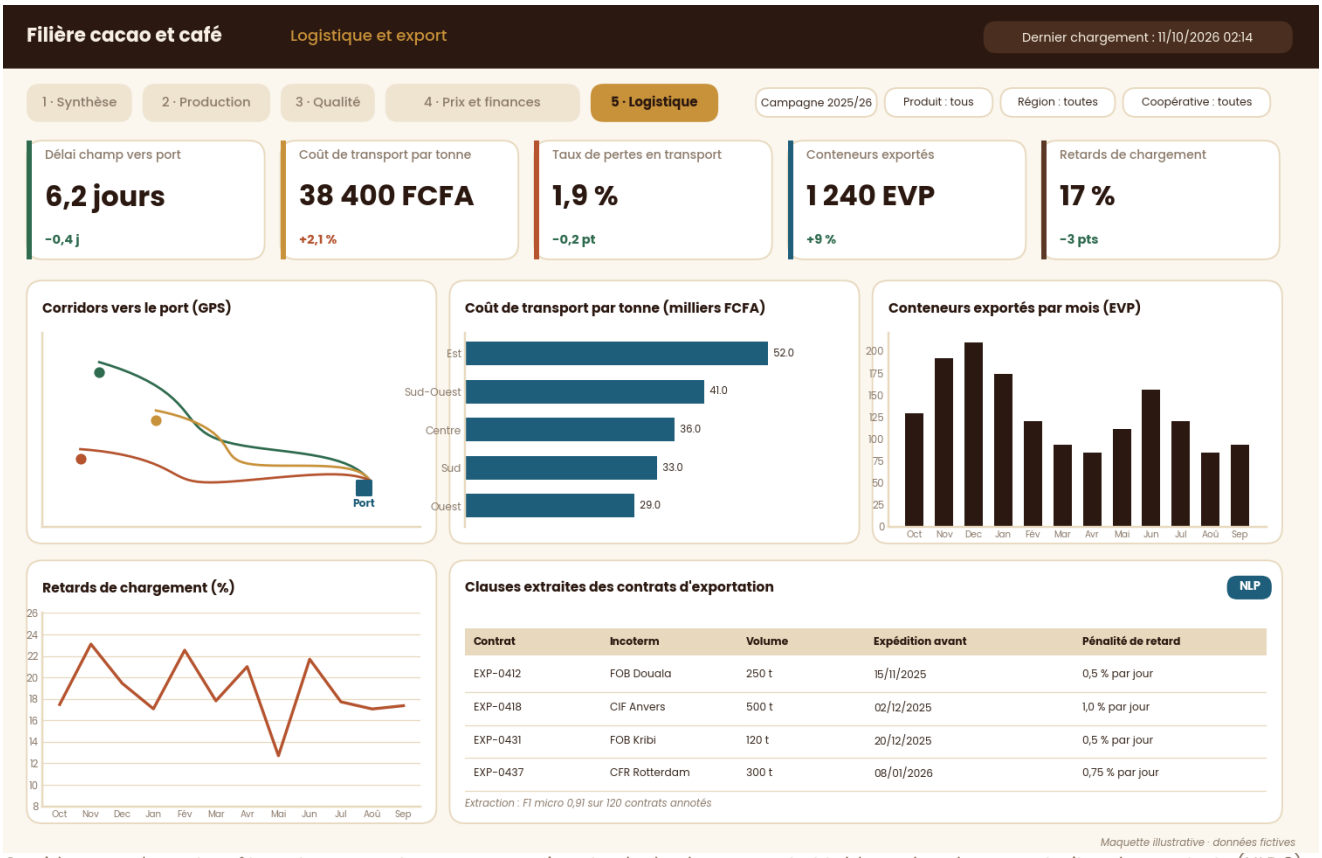
Distribution de l'humidité avec seuil, taux de refus par coopérative, grades par mois, matrice de confusion du modèle de rejet (ML 2) et défauts classés par le NLP (NLP 1).

Page 4 · Prix et finances



Prix mondial et prix producteur, prévision à trois mois (ML 3), paiements Mobile Money, délai par canal et simulation de marge par scénario (ML 3).

Page 5 · Logistique et export



Corridors vers le port, coût par tonne, conteneurs par mois, retards de chargement et tableau des clauses extraites des contrats (NLP 2).

Règles communes aux cinq pages

Exigence	Mise en œuvre
Filtres communs	Campagne, produit, région, coopérative, présents et synchronisés sur les cinq pages (Metabase : filtres de tableau de bord reliés à chaque carte ; Power BI : segments synchronisés ; Superset : filtres natifs)
Navigation	Onglets numérotés identiques en haut de chaque page (Metabase : onglets de tableau de bord ; Power BI : boutons de navigation par signet)
Actualité des données	En-tête de chaque page : « Dernier chargement : date et heure », alimenté par la requête sur etl.run_log vue au chapitre 8
ML et NLP	Chaque fonctionnalité sur sa page, avec son badge, ses métriques (MAPE, précision, rappel, F1) et la date de calcul du modèle
Lisibilité	Une couleur par produit, des unités sur chaque axe, des cartes d'indicateurs avec comparaison à la campagne précédente, pas plus de six visuels par page
Source unique	Toutes les cartes lisent le schéma dwh, ml ou nlp : jamais le staging ni les sources

Quel outil choisir ?

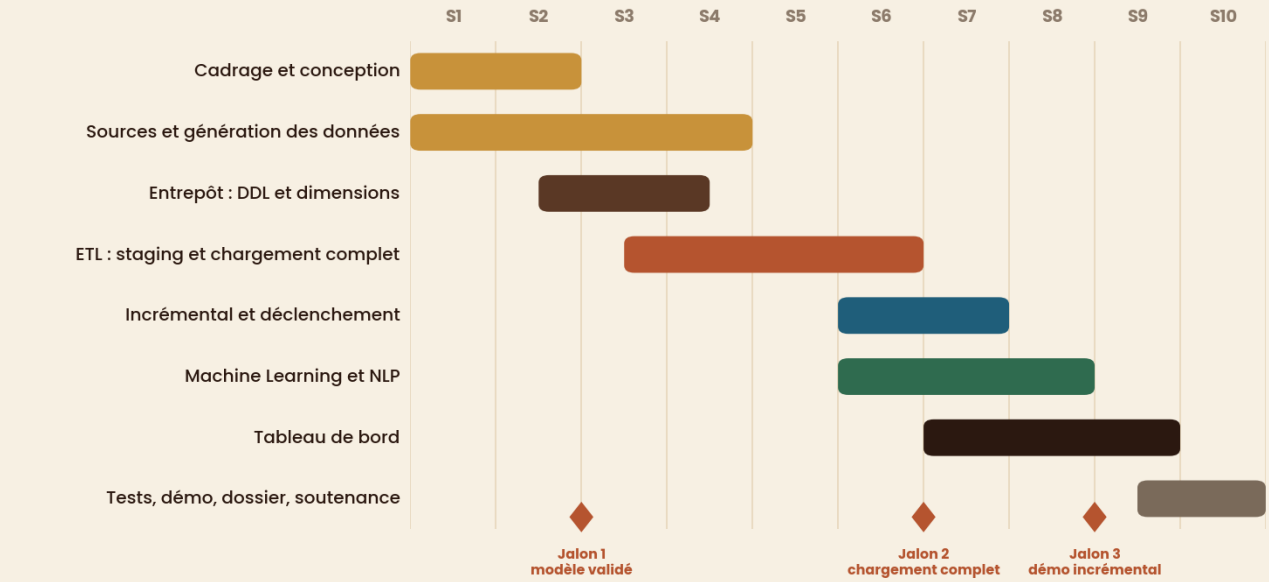
Metabase est le chemin le plus court : une image Docker, une connexion PostgreSQL, des questions en SQL ou en assistant, des onglets et des filtres reliés. Power BI est préférable si votre établissement le fournit et si vous maîtrisez DAX. Dans les deux cas, branchez-vous sur l'entrepôt, jamais sur les fichiers.

11 Plan de réalisation

Dix semaines, quatre rôles, un dépôt propre : l'organisation qui évite les mauvaises surprises.

Calendrier indicatif sur 10 semaines

À adapter à la date de soutenance fixée par l'enseignant



Calendrier indicatif : à comprimer ou à étirer selon votre date de soutenance.

Répartition des rôles (équipe de quatre)

Rôle	Responsabilités principales	À relire chez les autres
A · Sources et génération	Générateurs Faker, appels d'API, fichiers de référence, anomalies injectées, volumétrie	Règles de qualité
B · Modèle et ETL	Indicateurs, matrice, DDL de l'entrepôt, transformations, SCD2, chargement des faits	Générateurs (cohérence des clés)
C · Orchestration et qualité	Airflow, déclencheurs A, B et C, run_log, rejets, reprise sur incident, tests	Modèle de l'entrepôt
D · ML, NLP et BI	Cinq modèles, tables ml et nlp, tableau de bord de cinq pages, support de soutenance	Définitions des indicateurs

Les rôles organisent le travail, ils ne cloisonnent pas la connaissance : à la soutenance, n'importe quel membre doit pouvoir expliquer n'importe quelle partie. Planifiez une relecture croisée chaque semaine.

Arborescence du dépôt Git

Structure recommandée

```
projet9-cacao-cafe/
├── docker-compose.yml          # bases, Metabase (Airflow via le compose officiel)
├── README.md                   # démarrage en 5 commandes
├── docs/                       # dossier de conception : besoins, matrice, schémas, plan des flux
├── generate/                   # générateurs de données (livraisons.py, paiements.py, qualite.py, ...)
├── data/
│   ├── inbox/  archive/  quarantaine/      # dépôt, traités, en erreur
│   └── reference/                          # fichiers publics et replis d'API
├── sql/
│   ├── source/                 # schéma coop, déclencheurs
│   ├── dwh/                    # schémas stg, dwh, etl, ml, nlp, dim_date
│   └── indicateurs/            # une requête par indicateur (25 fichiers)
├── etl/                        # extract/  quality/  transform/  load/  common/
├── dags/                       # etl_filiere_cacao_cafe.py, etl_depot_fichier.py
├── ml/                         # prevision_tonnage.py, rejet_lot.py, prix_marge.py, nlp_defauts.py, nlp_contrats.py
├── dashboards/                 # export Metabase ou .pbix, captures d'écran
└── tests/                      # tests des règles de qualité, du SCD2 et de l'idempotence
```

Risques et parades

Risque	Parade
Une API est indisponible le jour de la démonstration	Cache et repli dans data/reference, avertissement dans run_log
Le portable ne supporte pas 1,7 million de lignes	Facteur --scale pour développer en petit, volume complet seulement pour la validation et la démonstration
Modèles trop parfaits ou trop mauvais	Génération probabiliste, référence simple, découpage temporel, explication des écarts
SCD2 ou suppressions mal testés	Tests automatiques sur le scénario des runs 0 à 4 avec comptages attendus
Fichiers Excel et CSV mal encodés	Lecture en UTF-8 explicite, séparateur et décimale configurés, contrôle R05
Dérive du périmètre	Socle d'abord (chapitre 2), options ensuite ; chaque ajout est validé par l'équipe
Licences et conditions d'usage des jeux publics	Citer la source, vérifier la licence, ne pas redistribuer les fichiers dans le dépôt public

12 Soutenance et validation finale

Un déroulé minuté, une démonstration qui ne dépend pas de la chance, et la liste de contrôle avant le jour J.

Déroulé conseillé (20 minutes)

Minutes	Contenu	Support
0 à 2	Contexte, questions de la direction, périmètre et hypothèses	Schéma de la chaîne de valeur
2 à 6	Architecture, 15 sources, modèle de l'entrepôt, grains et SCD2	Architecture, constellation, matrice
6 à 10	ETL : staging, règles de qualité, rejets, journal, reprise sur incident	Table etl.run_log, exemples de rejets
10 à 15	Démonstration en direct : runs incrémentaux 1 à 4, dépôt d'un fichier dans inbox, incident puis reprise	Terminal, Airflow, requête de contrôle
15 à 18	Tableau de bord : parcours des cinq pages, filtres communs, ML et NLP avec leurs métriques	Metabase ou Power BI
18 à 20	Limites, enseignements, perspectives	Une diapositive

Préparez un plan B de démonstration
Enregistrez une vidéo courte du scénario complet et gardez des captures d'écran de chaque étape. Si le réseau ou Docker flanche, vous poursuivez sans panique. Faites une répétition complète au moins deux fois, chronomètre en main.

Liste de contrôle avant le rendu

☐	**Sources**	15 sources lisibles, 4 formats au moins (7 prévus), 8 sources générées avec graine fixe et paramètre d'échelle
☐	**Volumétrie**	Plus d'un million de lignes au total, comptage par table prêt à être montré
☐	**Entrepôt**	Matrice, grains justifiés, constellation, dim_date, au moins un SCD2 (deux prévus), scripts DDL versionnés
☐	**ETL**	Staging, 10 règles de qualité, table des rejets, table de journalisation, gestion d'erreurs
☐	**Reprise**	Incident rejoué sans doublon, filigrane inchangé après l'échec
☐	**Incrémental**	Run 0 complet puis nouvelles lignes, lignes modifiées et lignes supprimées démontrées
☐	**Déclenchement**	Deux mécanismes au moins fonctionnels et montrés en direct (planification et dépôt de fichier)
☐	**Indicateurs**	25 indicateurs répartis 5 + 5 + 5 + 5 + 5, définis et vérifiés par requête SQL
☐	**ML et NLP**	3 ML et 2 NLP, calcul par lot planifié, métriques affichées, comparaison à une référence
☐	**Données textuelles**	Observations et clauses variées et réalistes, avec étiquettes pour l'évaluation
☐	**Tableau de bord**	5 pages, filtres communs, navigation, date du dernier chargement visible sur chaque page
☐	**Dossier**	Besoins, matrice des processus, modèle de l'entrepôt, plan des flux ETL, choix argumentés
☐	**Dépôt**	Scripts et jobs versionnés, README de démarrage, historique de commits lisible, aucun secret dans le dépôt

Les questions que le jury posera presque toujours

- Pourquoi cette granularité, et qu'est-ce qui se passe si on la change ?
- Comment garantissez-vous qu'un chargement rejoué ne crée pas de doublons ?
- Que devient un producteur qui change de coopérative, dans l'historique des tonnages ?
- Comment mesurez-vous la qualité de vos modèles, et face à quelle référence ?
- Comment le tableau de bord sait-il que les données sont à jour ?
- Qu'est-ce qui change si le volume est multiplié par dix ?

Bon courage

Un projet décisionnel réussi n'est pas celui qui a le plus de graphiques : c'est celui dont chaque chiffre affiché peut être expliqué, depuis la source jusqu'à la carte du tableau de bord. Gardez cette phrase en tête à chaque décision.